

Direct Tinygrad-to-Blackhole lowering through blackhole-py-rewrite

This report answers the archive-root `README.md`, which is the bundle's sole task specification (`README.md:3-5`). All citations are bundle-relative. No Tenstorrent device execution was attempted. "Implemented" below means executable source exists in the captured tree; it does **not** mean that this investigation independently established device correctness. The raw corpus was generated on the `PYTHON` device without realization or device execution (`metadata/SOURCE_STATE.md:96-104`).

Status labels used throughout:

- **[I] Implemented** in current executable source.
- **[E] Experiment** reproduced or tested in this CPU-only investigation.
- **[P] Proposed/planned** architecture or code not present in the source snapshot.
- **[H] Historical** implementation or design in `blackhole-py`, not the rewrite.
- **[N] Inference** from current code and artifacts; not a measured hardware result.
- **[U] Unverified** because required hardware, TT-Metal, TT-LLK, or other omitted evidence is unavailable.

The capture is intentionally a live-tree snapshot: `blackhole-py-rewrite` had six modified files, `blackhole-py` was ahead and heavily dirty/untracked, and Tinygrad was a clean checkout at commit `149fd91e221d4b2f4a23d6544627d8f4ffa0a2a6` (`metadata/SOURCE_STATE.md:3-16`). Live files, not stale document headers, are therefore the authority (`metadata/SOURCE_STATE.md:25-40`, `metadata/SOURCE_STATE.md:106-110`).

1. Executive recommendation

[P] Keep Tinygrad through `transform_to_call`, then invoke a pure backend graph lowerer from `lower_sink_to_linear` before generic `get_kernel_graph`. The lowerer should consume the complete callified inner `SINK`, inspect all top-level materialization roots, and either decline or return an ordinary Tinygrad graph composed of `CALL(PROGRAM, ...)`, `COPY`, `SLICE`, and `AFTER` nodes. Tinygrad should then continue through its existing dependency scheduler, parameter rebinding, memory planning where legal, JIT capture, `PROGRAM` cache, and runtime execution. This is the smallest seam that preserves current realization, assignment, alias, mutation, precompiled-function, and cache-normalization behavior while retaining shaped reductions, movement, broadcasts, and indexing for tile-aware lowering. The current code makes that seam exact: `Tensor.linear_with_vars` callifies first (tinygrad/tinygrad/tensor.py:178-182), and `lower_sink_to_linear` currently performs the single unconditional `create_schedule(get_kernel_graph(function)) call` (tinygrad/tinygrad/schedule/__init__.py:111-135).

[P] Do not introduce TTIR, a Tensor subclass, a TT-specific Tinygrad Op, or a scalar/GPU renderer as the compiler boundary. A short-lived analysis layer is sufficient: maps from UOps to logical/physical shape, tile access, effects, candidate regions, resources, core spans, and lowering recipes. The result of analysis is an immutable Blackhole bundle artifact, not another persistent graph IR. The current rewrite plan reaches the same broad conclusion and explicitly labels itself design-only (blackhole-py-rewrite/tenstorrent-lowering.md:1-14, blackhole-py-rewrite/tenstorrent-lowering.md:42-54); this report independently validates the boundary against the current Tinygrad source rather than accepting the document by age.

[P] Represent each fused TT region as one complete Tinygrad `PROGRAM` UOp whose payload is a serialized, immutable `TTBundleArtifact`. The artifact must contain the target/capability fingerprint, ordered tensor/scalar ports, per-core partition, five role images, CB/barrier/phase manifest, layout contract, mutations, and exact resource report. Its runtime callable binds concrete Tinygrad buffers and symbolic values, uploads/caches images, writes parameter tables, submits CQ commands, and reports completion. No new execution Op is needed: Tinygrad already permits `CALL(PROGRAM, ...)`, executes such calls through the device runtime, and uses `ProgramInfo` to carry globals, outputs, inputs, variables, auxiliary data, and launch values (tinygrad/tinygrad/uop/ops.py:1062-1071, tinygrad/tinygrad/uop/ops.py:1090-1135; tinygrad/tinygrad/engine/realize.py:176-186, tinygrad/tinygrad/engine/realize.py:244-281).

[P] Use a deterministic legality-and-partition model, not an opaque “fusion heuristic.” Candidate fusion is legal only when semantic effects, tile mappings, partial-tile masks, layouts, external ports, CB lifetimes, L1, per-role text/local memory, `Dst/SFPU/replay` use, NoC streams, core topology, and phase ownership all fit a versioned capability table. Among legal plans, choose lexicographically by program count, external DRAM bytes, layout-conversion bytes, NoC bytes, peak L1, maximum role code, and a stable structural signature. When a candidate fails, enumerate legal cut edges and choose a deterministic minimum-cost cut; diagnostics must name the exact resource or semantic rule.

[N] The current 14-call Llama block is not evidence that TT needs 14 programs, and it is not evidence that one program is possible. Current generic Tinygrad already fuses matmul epilogues, projections with normalization factors, RoPE with neighboring work, a KV mutation with producer computation, and residual additions. The 14 calls are dominated by reduction/softmax materializations plus one explicit model `CONTIGUOUS` boundary. A host-only perturbation removed the FFN gate's explicit `contiguous()` and reduced the schedule from 14 to 13 calls; removing only the outer result `contiguous()` did not change the count (research-output/fusion_boundary_experiment.out:1-12). A credible TT target is initially **five to eight programs per decode block**, but that range is a design hypothesis—not a hardware result—and the resource verifier must be allowed to produce more.

[P] The shortest credible implementation path is:

1. Land the generic post-callify graph-lowering hook and tests; a working patch in `research-output/architecture_hook.diff` passes four dedicated hook tests and six selected

assignment/precompiled/custom-kernel regressions (research-output/architecture_hook_test.out:1-9, research-output/architecture_hook_selected_regression.out:1-14).

2. Harden the rewrite's runtime ABI. The attached Program.bind patch rejects foreign parameters, dtype/location/shape/padded-shape mismatches, and non-32-bit addresses; all four tests pass (research-output/first_patch.diff:1-65, research-output/first_patch_test.out:1-9).
3. Add TT device plumbing with an inert renderer only because current pm_compile still requests a renderer even for an already complete PROGRAM; do not put lowering in that renderer (tinygrad/tinygrad/engine/realize.py:244-247).
4. Generate the existing one-tile add1 bundle from shaped UOps, then add multi-tile/broadcast elementwise, reduction/RMSNorm, matmul+epilogue, RoPE/KV, GQA/softmax, SwiGLU/MLP, one block, and finally full decode/prefill.

2. Evidence/status table: implemented, planned, historical, unverified

Area or claim	Status	Source-grounded finding
Rewrite assembler and per-role kernel builder	[I]	KernelBuilder has a fixed role and core, role-local allocator ranges, register/scope machinery, a firmware mode, and one-shot lowering; NoC access is role-restricted (blackhole-py-rewrite/asm.py:12-20, blackhole-py-rewrite/asm.py:206-239).
Five-role bundle construction	[I]	KernelBundle accepts BRISC, NCRISC, TRISC0, TRISC1, and TRISC2 functions, creates a fresh builder per core/role, and lowers every role to bytes (blackhole-py-rewrite/program.py:171-207).
One-tile BF16 add1 bundle	[I] [E]	The example generates reader/unpack/math+SFPU/pack/writer roles with two CBs and a barrier (blackhole-py-rewrite/examples/add1.py:20-71). Host-only lowering produced all five images and the expected parameter/CB manifest (research-output/add1_host_lowering.out:1-8).
Rewrite Program, upload, launch, CQ, firmware	[I]	Program groups identical images, checks fixed text partitions, uploads chunks, writes initial parameters, clears barriers, and appends launch commands (blackhole-py-rewrite/program.py:82-138). CQ serializes unicast/multicast/run records and synchronously waits for a completion event (blackhole-py-rewrite/cq.py:78-146, blackhole-py-rewrite/cq.py:223-244). Firmware dispatches the five workers through fixed role entrypoints (blackhole-py-rewrite/fw/brisc.py:27-79, blackhole-py-rewrite/fw/ncrisc.py:4-20, blackhole-py-rewrite/fw/trisc.py:6-28).
Rewrite device execution path	[I], not device-tested here	Device initializes PCIe, DRAM, firmware, and CQ, creates DRAM-transfer programs, converts host arrays to/from physical tile/face order, and submits each Program (blackhole-py-rewrite/device.py:11-47, blackhole-py-rewrite/device.py:50-123).
Rewrite CB producer/consumer API	[I], limited	CB reset/local interface and reserve/wait/push/pop operations exist (blackhole-py-rewrite/ttk/cb.py:4-29, blackhole-py-rewrite/ttk/cb.py:45-112). The TODO's statement that no real CB API exists is stale (blackhole-py-rewrite/todo.md:36-55).
Rewrite SFPU API	[I], very limited	The code tracks eight LRegs, initialization/ownership hazards, and supports load/store, add-immediate, advance, replay installation, and tile execution (blackhole-py-rewrite/ttk/sfpu.py:63-153). The TODO's blanket "no SFPU" statement is stale (blackhole-py-rewrite/todo.md:201-222). It still lacks the operation set needed for RMSNorm, RoPE, softmax, and SwiGLU.
Rewrite matmul/reduction/high-level compiler recipes	[P]/missing	Math currently initializes, copies SrcA to Dst, exposes SFPU, and publishes Dst; there is no matmul or reduction template (blackhole-py-rewrite/ttk/math.py:6-44). Unpack and Pack cover a narrow one-source/one-tile path (blackhole-py-rewrite/ttk/unpack.py:12-50, blackhole-py-rewrite/ttk/pack.py:5-48).
Rewrite direct Tinygrad backend	[P]/missing	The current design file says it is not implemented (blackhole-py-rewrite/tenstorrent-lowering.md:1-9), and the captured Tinygrad source has no TT backend/runtime (metadata/SOURCE_STATE.md:96-104).
Rewrite Program.bin validates ABI	Documented but not implemented in snapshot	Current code only writes a 32-bit address (blackhole-py-rewrite/program.py:94-101), while TTK.md claims binding enforces compatibility (blackhole-py-rewrite/TTK.md:55-65). The attached tested patch closes this gap (research-output/first_patch.diff:1-65).

Area or claim	Status	Source-grounded finding
Rewrite KernelBuilder.standalone	Documented, not implemented	TTK.md names it (blackhole-py-rewrite/TTK.md:67-70), but current KernelBuilder exposes firmware and ordinary lowering, not standalone (blackhole-py-rewrite/asm.py:206-239).
Exact TTK state/CFG merge	[P]/partial	Current state tracks pipe/context/MOP/SFPU and suppresses redundant writes, but unknown CFG values default to zero and there is no control-flow merge (blackhole-py-rewrite/ttk/tensix.py:182-215, blackhole-py-rewrite/ttk/tensix.py:247-275). The richer exact-state/merge model is design text (blackhole-py-rewrite/TTK.md:86-132).
Historical blackhole-py runtime	[H] implemented	It has per-core role specialization, RTAs, CB/semaphore/kernel layout and overlap checks (blackhole-py/program.py:216-277, blackhole-py/program.py:311-391), plus slow dispatch, ordered fast batching, and capture/replay (blackhole-py/device.py:82-150, blackhole-py/device.py:197-216; blackhole-py/cq.py:392-439).
Historical hand-built Llama path	[H] implemented in source	The old tree contains a sequential full-model runner whose embedding, blocks, KV caches, final norm, vocabulary projection, and argmax are device-side (blackhole-py/examples/llama3/model.py:1-12). A block explicitly invokes RMSNorm, attention, residual, RMSNorm, MLP, residual as several programs (blackhole-py/examples/llama3/block.py:39-149).
Historical fused SwiGLU	[H] implemented in source	One launch consumes gate/up and keeps SiLU intermediate out of CB/DRAM; sharding is deterministic (blackhole-py/examples/llama3/swiglu.py:1-11, blackhole-py/examples/llama3/swiglu.py:283-357).
Historical TTIR/pre-callify compiler	[H] design only	TTIR.md explicitly describes an implementation that is still to be built (blackhole-py/TTIR.md:1-35) and proposes interception before callify (blackhole-py/TTIR.md:455-519). There is no TTIR implementation in the snapshot.
Raw UOp corpus	[I] [E]	The generator records shaped, callified, kernel-graph, scheduled, base-AST, and optional late Python stages (uop-dumps/generate_uop_dumps.py:147-188). All renderer-neutral stages and call ABI for the five central probes regenerated byte-for-byte in full probe order (research-output/uop_renderer_neutral_regeneration.out:1-6).
TT-Metal/TT-LLK-derived hardware claims	[U]	Those repositories and standalone ISA documentation are intentionally absent (metadata/EXCLUSIONS.md:23-31). Claims such as a 64-entry Blackhole CB namespace, exact Dst capacity, or exact reset-state behavior cannot be promoted to facts from this bundle alone.
Device correctness/performance of either Blackhole tree	[U] in this investigation	Hardware execution was prohibited by the task (README.md:29-31). Source existence and host-side lowering are established; silicon correctness, numerical accuracy, throughput, and queue behavior were not revalidated here.

Two code/document conflicts deserve immediate operational treatment. First, the real parameter table begins at 0x4240 with 0xEC0/4 = 944 slots (blackhole-py-rewrite/fw/consts.py:3-16), while TTK.md names 0x4100 (blackhole-py-rewrite/TTK.md:55-65). Second, current CB and SFPU APIs exist even though the TODO still lists them as absent. Compiler work should use executable modules and tests as authority, then update the documents.

3. Current architecture and the exact Tinygrad seam

3.1 Current Tinygrad path from Tensor UOps to execution

[I] Tensor construction remains shaped. Tensor operators call `_apply_uop`, which invokes a UOp-building function, attaches metadata, and wraps the new UOp without scheduling (`tinygrad/tinygrad/tensor.py:108-117`). At this level, the graph still contains shaped `REDUCE`, `RESHAPE`, `PERMUTE`, `EXPAND`, `PAD`, `SHRINK`, broadcasts, symbolic values, and explicit `CONTIGUOUS`/mutation structure.

[II] Realization callifies before scheduling. `Tensor.linear_with_vars` builds one `SINK` from all requested outputs, calls `transform_to_call`, updates live Tensor identities from the returned map, and then schedules the callified graph (`tinygrad/tinygrad/tensor.py:178-182`). `Tensor.realize` selects unrealized device tensors and executes that linear graph (`tinygrad/tinygrad/tensor.py:190-195`). `Tensor.callify` likewise replaces every affected live Tensor with the buffer state after the resulting call (`tinygrad/tinygrad/tensor.py:172-176`).

`transform_to_call` is not a cosmetic wrapper. It provides the backend-independent semantic contract the TT path should retain:

- Only explicit materialization/effect roots—`COPY`, `AFTER+STORE`, `CONTIGUOUS`, and their tagged parents—are marked (`tinygrad/tinygrad/callify.py:32-42`).
- Device-backed `CONTIGUOUS` becomes a fresh buffer plus `STORE` and `AFTER` (`tinygrad/tinygrad/callify.py:44-52`).
- A precompiled `FUNCTION` makes non-effect inputs contiguous, allocates/redirects outputs, builds a multi-output inner `SINK`, creates an opaque `CALL`, and returns each output through `AFTER`; symbolic outputs are shrunk to the resolved caller shape (`tinygrad/tinygrad/callify.py:101-142`).
- Finalization records assignments and maps original live UOps to their realized buffer identity (`tinygrad/tinygrad/callify.py:169-181`).
- Concrete `BUFFER`, `SLICE`, and `BIND` leaves become ordered `PARAMS`; the bound value is stripped so different symbolic values can share the schedule cache (`tinygrad/tinygrad/callify.py:183-202`).

The resulting shape is conceptually:

```
CALL(
  SINK(
    AFTER(output_buffer, STORE(output_buffer, shaped_expression), ...effects...),
    AFTER(mutated_cache, STORE(cache_view, update), ...),
    ...multi-output roots...
  ),
  actual_input_buffer_0,
  actual_input_buffer_1,
  actual_output_buffer_0,
  symbolic_bindings,
  ...
)
```

[I] lower_sink_to_linear is the exact dispatch seam. `pm_schedule` rewrites a callified inner `SINK` through `lower_sink_to_linear` (`tinygrad/tinygrad/schedule/__init__.py:137-145`). The current function uses `function.key` as its cache key, then unconditionally calls `get_kernel_graph(function)` and `create_schedule(...)` (`tinygrad/tinygrad/schedule/__init__.py:111-135`). There is no earlier current backend hook and no need to intercept individual Tensor methods.

[I] Generic `get_kernel_graph` is already policy, not just canonicalization. Its path performs multi-device/movement resolution, rangeification, symbolic/reduction simplification, stage/buffer insertion/removal, buffer-limit handling, and kernel splitting (tinygrad/tinygrad/schedule/rangeify.py:574-619). That is useful for CPU/GPU backends, but its scalar ranges, generic fusion costs, and buffer rules are not a TT tile/core/CB planner. The direct TT lowerer should selectively reuse safe algebraic and movement utilities, not invoke this orchestration and attempt to reconstruct tile semantics afterward.

[I] `create_schedule` remains valuable after TT lowering. It splits AFTER dependencies, finds reader/writer relationships, adds explicit write-after-read constraints, topologically orders kernel calls, and returns LINEAR (tinygrad/tinygrad/schedule/__init__.py:9-85). This is precisely why the TT lowerer should emit ordinary calls and effects rather than a custom execution path.

[I] Generic codegen turns each scheduled call into PROGRAM. `to_program` recognizes an already complete four-source PROGRAM and bypasses ordinary shaped rewrite; otherwise it progressively rewrites SINK through lowered SINK, LINEAR, SOURCE, and BINARY forms (tinygrad/tinygrad/codegen/__init__.py:180-257). `pm_compile` currently matches both `CALL(SINK)` and `CALL(PROGRAM)` and requests `Device[...].renderer` before `to_program` (tinygrad/tinygrad/engine/realize.py:244-247). A TT device therefore needs a renderer-shaped plumbing object, but it should be inert for a complete TT PROGRAM.

[I] Runtime execution already has the right opaque call contract. `exec_kernel` resolves call parameters to Tinygrad buffers, selects the device runtime from the PROGRAM, computes launch values, and calls the runtime with ordered buffers and vals (tinygrad/tinygrad/engine/realize.py:176-186). `run_linear` compiles/links and executes each call in the LINEAR schedule (tinygrad/tinygrad/engine/realize.py:263-281). The TT runtime can fit this contract without a new Op.

3.2 What is present at each UOp dump stage

[I] Stages 00–50 are renderer-neutral primary evidence; 60–70 are intentionally late Python comparisons. The generator constructs stage 00 shaped SINK, stage 10 callified CALL, stage 11 callified inner SINK, stage 20 generic kernel graph, stage 30 dependency schedule, stage 40 final resolved/memory-planned schedule, and one stage 50 base AST per call (uop-dumps/generate_uop_dumps.py:147-188). Its manifest states no realization or device execution and labels 00–50 as the primary evidence (uop-dumps/generate_uop_dumps.py:333-350).

Stage	What TT can still see	Architectural use
00_shaped_sink	Maximum model intent, concrete buffers, movement, mutation, explicit CONTIGUOUS; no normalized call ABI.	Evidence and debugging, not the backend boundary.
10_callified_call	Opaque call wrapper, ordered actual arguments, normalized inner function.	Runtime/cache ABI evidence.
11_callified_inner_sink	Shaped reductions/movement/broadcast/indexing plus explicit output/mutation roots and PARAMS.	Recommended TT compiler input.
20_kernel_graph	Generic rangeification, materialization, and kernel split decisions already applied.	Comparison and fallback; too policy-laden as the direct TT input.
30/40	Ordered ordinary calls and allocated/rebound buffers.	Preserve after TT emits calls; too late to choose TT graph fusion.
50	Per-call scalar/range AST still often recognizable as matmul, reduction, and softmax.	Recipe validation and fallback kernel work, but graph-level fusion is already lost.
60/70	Python renderer's late lowered/linear form.	Negative comparison only.

The historical claim that post-rangeify code is merely flat STORE/INDEX with no recognizable matmul or reduction is too absolute for this Tinygrad commit. Stage 50 still contains an explicit K-axis REDUCE(MUL(. . .)), bias add, and ReLU WHERE for matmul epilogue (uop-dumps/current/matmul_epilogue/50_kernel_00_base_ast.txt:4-34), and explicit reductions remain in RMSNorm and attention. What is lost after stage 11 is not every operation name; it is the freedom to choose graph-level materialization, tiling, multi-role phases, and cross-kernel fusion.

3.3 Current rewrite path: `KernelBuilder` → `KernelBundle` → `Program` → `Device/CQ`

[I] `KernelBuilder` is a single-core, single-role assembler context. The five roles and their fixed local-memory ranges are declared centrally (blackhole-py-rewrite/asm.py:12-20). Each builder owns register/allocation/state and can access the role-appropriate TTK interfaces; lowering is one-shot (blackhole-py-rewrite/asm.py:206-239).

[I] `KernelBundle` is the current compiler-facing composition object. It takes an ordered core set, ordered Params, and up to five role functions (blackhole-py-rewrite/program.py:171-180). It allocates CB IDs in insertion order, enforces positive depth, a 32-CB software limit, and placement inside the L1 data-buffer region (blackhole-py-rewrite/program.py:183-190). During lower, it creates a fresh `KernelBuilder` for every (core, role), invokes the role function if present, and stores the lowered bytes (blackhole-py-rewrite/program.py:196-207).

A subtle but important point is omitted versus empty roles. `Program.kernel` can synthesize a one-instruction return if a manually constructed `Program` lacks an image (blackhole-py-rewrite/program.py:90-92). `KernelBundle.lower`, however, creates a builder and stores an image for every role even when the function is `None` (blackhole-py-rewrite/program.py:199-205). This is safe against stale worker text because every role gets a valid return path, but the artifact currently does not preserve the semantic distinction between “intentionally disabled,” “empty return,” and “missing due to compiler error.” **[P]** The bundle manifest should record an explicit role state while still uploading a return image for every selected core.

[I] `Program` is an immutable upload/launch description, but its binding ABI is under-specified. It stores per-core/per-role bytes, named params, CBs, barriers, and launch commands (blackhole-py-rewrite/program.py:82-88). It groups identical role images for multicast, chunks writes, checks each fixed text partition, writes the initial parameter table, clears barriers, and appends launch commands (blackhole-py-rewrite/program.py:103-138). Current bind only writes a replacement address (blackhole-py-rewrite/program.py:94-101); it does not establish that the replacement has the same location, dtype, logical shape, padded shape, or future layout. The tested first patch adds those checks.

[I] The five roles execute concurrently under firmware control. BRISC coordinates launch and waits for subordinate completion before notifying dispatch (blackhole-py-rewrite/fw/brisc.py:27-79). NCRISC and TRISCs wait for GO, call their fixed worker-text entrypoints, and mark completion (blackhole-py-rewrite/fw/ncrisc.py:4-20, blackhole-py-rewrite/fw/trisc.py:6-28). Role-local text partitions, the parameter table, and the data-buffer boundary are fixed and validated (blackhole-py-rewrite/fw/consts.py:3-16, blackhole-py-rewrite/fw/consts.py:71-83).

[I] `Device` and `CQ` currently execute one rewrite `Program` at a time. Device initialization creates PCIe/DRAM/queue state and uploads/boots firmware (blackhole-py-rewrite/device.py:11-47). `Device.run` prepends any required DRAM-transfer program and submits each program synchronously (blackhole-py-rewrite/device.py:112-123). `CQ` supports aligned PAD, unicast write, multicast write, and run records; `submit` rewrites the final run with a new event, publishes all records, and blocks in wait (blackhole-py-rewrite/cq.py:23-27, blackhole-py-rewrite/cq.py:223-244).

[H] The predecessor has useful runtime capabilities but is not the compiler architecture. Its `Program` has per-core sender/receiver role specialization, RTAs, CB/semaphore layout, and overlap checks (blackhole-py/program.py:216-277, blackhole-py/program.py:311-391). Its device can queue dependent programs, lower all of them, submit one ordered fast-dispatch stream, and capture/replay traces (blackhole-py/device.py:82-150, blackhole-py/device.py:209-216; blackhole-py/cq.py:427-439). These are candidates to port into the rewrite after the new immutable bundle ABI is stable; the old ad hoc kernel constructors and runtime argument conventions should not become the Tinygrad compiler surface.

3.4 Three distinct meanings of “one program”

The task correctly separates three contracts (README.md:121-137):

1. **[I] One Blackhole bundle/program:** one selected core set, with specialized BRISC/NCRISC/TRISC0/TRISC1/TRISC2 images, CBs, barriers, parameters, and launch protocol.
2. **[P] One Tinygrad PROGRAM UOp:** one opaque executable artifact plus its ordered buffer/scalar ABI, called through ordinary Tinygrad CALL and observed through AFTER.
3. **[P] One fused graph region:** a compiler decision. A block may require several bundle programs because of effects, external materialization, L1/CB/Dst/code/SFPU/NoC limits, incompatible core partitions, or layout conversions.

Combining role kernels means compiling five cooperating instruction streams and a communication/phase plan into one artifact. It does not mean concatenating instruction streams, and graph fusion must stop when the deterministic verifier says the region does not fit.

4. Proposed end-to-end architecture and call graph

4.1 Compiler and runtime layers

[P] The end-to-end path should be:

```
Tinygrad Tensor methods
-> shaped UOp DAG
-> transform_to_call
    - explicit STORE/AFTER effects
    - precompiled multi-output CALL semantics
    - PARAM ordering and symbolic cache normalization
-> generic backend-graph-lowering registry
    - request contains the whole callified SINK and every top-level root
    - TT lowerer claims only supported TT root sets
-> TT shaped-graph analysis (short-lived records, no TTIR)
    - normalize movement and logical indexing
    - infer logical shape, padded tile shape, masks, layouts, effects
    - recognize recipes and enumerate candidate fused regions
    - assign deterministic core/tile spans
    - verify resources and split deterministically
-> TTK templates + KernelBuilder
    - emit one fresh builder per core and role
    - allocate CBs/barriers/local scratch
    - produce exact code/resource manifest
-> immutable TTBundleArtifact
-> ordinary Tinygrad PROGRAM + CALL + AFTER graph
-> create_schedule
    - RAW/WAR/effect dependencies and topological order
-> cached PARAM resolution / buffer allocation / supported memory planning
-> pm_compile (complete PROGRAM is preserved; inert TT renderer is plumbing only)
-> TT runtime callable
    - validate/bind buffers and scalars
    - cache/upload images
    - enqueue ordered commands and wait or return timing
```

This division gives each layer one authority:

- Tinygrad callify owns realization identity, assignment, mutation, precompiled outputs, and parameter normalization.
- The TT lowerer owns tile semantics, recipe matching, fusion, resources, partitioning, and bundle construction.
- TTK owns correct low-level engine operations and exact resource/state effects.
- KernelBundle/Program own immutable executable packaging.
- Tinygrad create_schedule owns inter-program ordering.
- The TT runtime owns physical buffers, binding, upload caching, CQ submission, and synchronization.

4.2 Root selection: inspect materializations, not `uop.device`

[I] `uop.device` is unsuitable as the dispatch authority. After special cases it returns the first device found while walking sources (`tinygrad/tinygrad/uop/ops.py:755-769`). A callified `SINK` can contain several outputs, mutations, copies, and devices. Selecting from one arbitrary result can route a mixed root set incorrectly.

[P] The registry should pass all top-level `function.src` roots to each lowerer. TT then classifies each root after peeling only semantics-preserving wrappers:

```
COMPUTE_OUTPUT
  AFTER(output PARAM, STORE(output PARAM/view, shaped value), dependencies...)

MUTATION
  AFTER(persistent PARAM/view, STORE(persistent view, update), dependencies...)

COPY
  AFTER(destination, COPY(source)) or a root CALL(COPY, ...)

VIEW/ALIAS
  a result whose identity is a slice/view of an existing buffer and needs no TT compute
```

The first implementation should accept one of two root sets:

1. all TT compute/mutation outputs; or
2. TT compute/mutation plus ordinary ingress/egress `COPY` roots that remain generic Tinygrad calls.

It should reject unrelated mixed-device compute with a diagnostic listing root index, effect kind, device set, and unsupported crossing. Later, the generic registry may partition independent top-level roots by backend and merge the returned ordinary call graphs before `create_schedule`.

4.3 Direct shaped-UOp analysis without TTIR

[P] The lowerer should keep immutable records keyed by original UOp, not clone the graph into another SSA. A useful minimum record is:

```
@dataclass(frozen=True)
class ValueInfo:
    uop: UOp
    logical_shape: tuple[sint, ...]
    physical_shape: tuple[sint, ...]
    dtype: DType
    tile_map: TileMap
    valid_mask: TileMask
    layout: Layout
    effect: Effect
    aliases: frozenset[BufferIdentity]
    users: tuple[UOp, ...]
```

Normalization should canonicalize views into an affine or guarded tile-access description while retaining the original UOp for diagnostics. Logical-only `RESHAPE/PERMUTE/EXPAND` can be folded into `TileMap`; `PAD` and partial `SHRINK` contribute validity masks and reduction identities; `FLIP` contributes a signed stride; gathers/non-affine tensor indexing become explicit indexed-access recipes or unsupported diagnostics. Physical retilization is not silently treated as a free view.

Pattern recognition should be recipe-based and compositional:

- primitive elementwise expressions and casts;
- row/tile reductions;
- matmul from `REDUCE(ADD, MUL(...))` plus index domains;
- RMSNorm from square-reduce, scale, epsilon, reciprocal-sqrt, and gamma multiply;
- stable softmax from max, shifted exponent, sum, reciprocal, normalize;
- RoPE from paired half/complex-style rotation and position/frequency lookup;
- SwiGLU from gate projection, SiLU, up projection, and product;
- Q/K/V projections, GQA head mapping, causal/length masks, score reduction, online or materialized softmax, value reduction;
- KV update from symbolic indexed `STORE` into persistent cache state.

A recipe does not immediately force fusion. It contributes semantic requirements, alternative tile algorithms, resource formulas, supported layouts/dtypes, numerical mode, and emitters. The planner composes recipes into region candidates, then the verifier decides whether each candidate is one program.

4.4 Historical pre-callify/TTIR proposal versus current post-callify lowering

`blackhole-py/TTIR.md` proposes a tile-SSA IR and interception before `transform_to_call` (`blackhole-py/TTIR.md`:455-519). It also has valuable ideas: fusion tiers, explicit resource budgets, and a state transition model described as requirements/body/effects (`blackhole-py/TTIR.md`:306-376). But it says implementation remains future work (`blackhole-py/TTIR.md`:1-35).

Question	Historical pre-callify + TTIR	Recommended post-callify direct lowering
Realization/materialization identity	Backend must reproduce Tinygrad's tagging, buffer creation, and live-Tensor update semantics.	Reuses current <code>transform_to_call</code> , including <code>CONTIGUOUS → STORE/AFTER</code> and the returned live-buffer map (<code>tinygrad/tinygrad/callify.py</code> :32-52, <code>tinygrad/tinygrad/tensor.py</code> :178-182).
Assignment and KV mutation	Must rediscover write targets, ordering, alias identity, and write-after-read behavior.	Receives explicit <code>STORE/AFTER</code> ; later <code>create_schedule</code> already adds WAR dependencies (<code>tinygrad/tinygrad/schedule/__init__.py</code> :21-85).
Precompiled functions/multiple outputs	Must duplicate output allocation/redirect/copy/shrink rules.	Receives the normalized multi-output inner <code>SINK</code> and call ABI (<code>tinygrad/tinygrad/callify.py</code> :101-142).
Symbolic cache normalization	Must define a parallel cache ABI.	Reuses <code>BUFFER/SLICE/BIND → PARAM</code> , including stripped bound values (<code>tinygrad/tinygrad/callify.py</code> :183-202).
Shape and operation intent	Maximum raw intent.	Stage 11 still preserves shaped reductions, movement, broadcasts, symbolic indices, and effects; enough for TT recipes (<code>uop-dumps/generate_uop_dumps.py</code> :154-168).
Fusion freedom	Maximum.	Still before generic rangeify/materialization, so graph-level TT fusion remains available.
Compiler data model	New persistent tile SSA plus conversion and verification infrastructure.	Short-lived analysis records over canonical Tinygrad UOps.
Inter-program schedule/JIT/runtime	Must reconnect custom IR to Tinygrad.	Emits ordinary UOps and reuses existing schedule/JIT/runtime contracts.
Maintenance surface	Large fork before a rapidly evolving semantic pass.	One generic hook plus TT modules; default path remains unchanged.
Verdict	Salvage resource/state concepts; do not implement as architecture.	Recommended.

The strongest historical objection to a later seam is that generic lowering erases operation intent. That is correct for stages 60/70 and partly correct after stage 20, but not for the proposed stage 11 boundary. Even stage 50 still exposes recognizable matmul and reductions, so stage 11 plainly has sufficient structural information. Conversely, callify's mutation and cache work is real current code, not incidental historical policy. Reimplementing it would create correctness risk with no demonstrated benefit.

4.5 One TT region to one ordinary Tinygrad graph

[P] Each compiled region should return the same kinds of nodes that generic rangeify would eventually produce:

```
program_0 = PROGRAM(...TTBundleArtifact_0...)
call_0    = CALL(program_0, output_0, input_0, weight_0, cache, start_pos, ...)
state_0   = AFTER(output_0, call_0)
cache_1   = AFTER(cache, call_0)          # mutation result / dependency token

program_1 = PROGRAM(...TTBundleArtifact_1...)
call_1    = CALL(program_1, output_1, state_0, cache_1, ...)
result    = AFTER(output_1, call_1)

SINK(result, cache_1, generic_copy_after_if_needed, ...)
```

For one call that mutates and produces multiple outputs, all affected buffer states reference the same call token. This lets `create_schedule` see that the call is the producer and preserves ordering without inventing a TT execution Op. The semantic `SINK` retained as a `PROGRAM` source also permits inspection, `ProgramInfo.from_sink`, and future validation tools.

4.6 Compilation and runtime cache boundaries

[P] Cache identity must be stronger than `function.key`. The generic schedule cache currently uses only `function.key` (`tinygrad/tinygrad/schedule/__init__.py:111-124`). A TT-lowered graph also depends on:

```
Tinygrad callified function key
+ backend graph-lowerer name/version
+ target chip and harvested topology/capability hash
+ bundle ABI version
+ layout policy version
+ recipe/template source fingerprints
+ dtype/numerical mode
+ deterministic partition policy version
+ any compile-time symbolic maxima
```

Runtime buffer addresses and current `start_pos` values must **not** enter the compile key. They are launch bindings. Compile-time symbolic ranges/maxima do enter the key if they change code, memory, masks, or partitioning.

Use three caches:

1. **Graph-lowering cache:** callified function key plus lowerer/capability/layout/recipe fingerprint → ordinary TT call graph and bundle artifacts.
2. **Artifact/image cache:** artifact content hash → per-role bytes and manifest; shared across programs with identical images.

3. **Runtime upload cache:** (device instance, artifact hash, core/image placement) → uploaded state.

Parameter/scalar tables are rebound for each launch.

The historical rewrite groups identical image bytes for multicast (`blackhole-py-rewrite/program.py:103-128`); the same content identity should drive the runtime upload cache.

5. Concrete APIs and UOp-level pseudocode

5.1 Minimal generic Tinygrad hook

[E] A working minimal patch is included in `research-output/architecture_hook.diff`. It adds `tinygrad/tinygrad/schedule/backend.py`, imports it from `tinygrad/tinygrad/schedule/__init__.py`, and changes only `lower_sink_to_linear`. The default path retains `function.key` and the exact existing `get_kernel_graph` call. A claiming backend returns an ordinary graph plus a non-empty cache tag; the hook hashes lowerer name and tag into the schedule key. Lowerers are tried in sorted name order, and multiple claims fail deterministically. Four focused tests pass (`research-output/architecture_hook_test.out:1-9`). Selected tests exercising AFTER/STORE, nested/precompiled functions, implicit outputs, and multi-output custom kernels also pass (`research-output/architecture_hook_selected_regression.out:1-14`).

The tested API is intentionally small:

```
@dataclass(frozen=True)
class GraphLoweringRequest:
    function: UOp          # complete callified inner SINK
    roots: tuple[UOp, ...] # every top-level materialization/effect root

@dataclass(frozen=True)
class BackendGraph:
    graph: UOp          # ordinary call/effect graph accepted by create_schedule
    cache_tag: bytes     # backend/capability/ABI/layout/recipe discriminator

GraphLowerer = Callable[[GraphLoweringRequest], BackendGraph | None]

def register_graph_lowerer(name: str, lowerer: GraphLowerer) -> None: ...
def unregister_graph_lowerer(name: str) -> None: ...
def try_lower_graph(function: UOp) -> tuple[str, BackendGraph] | None: ...
```

Production refinement should make registration happen through normal device/backend module discovery rather than application-side global mutation, but it must remain pure: scheduling must not call `device[...]`, because device lookup dynamically imports and constructs the runtime (`tinygrad/tinygrad/device.py:14-35`). The selector should be capability metadata, not an open PCIe device.

A fuller request/result surface can evolve without changing the seam:

```
@dataclass(frozen=True)
class MaterializationRoot:
    index: int
    value: UOp
    effect: Literal["compute", "copy", "mutation", "view"]
    devices: tuple[str, ...]
    target: UOp | None

@dataclass(frozen=True)
class GraphLoweringRequest:
    function: UOp
```

```

    roots: tuple[MaterializationRoot, ...]
    normalized_key: bytes

@dataclass(frozen=True)
class BackendGraph:
    graph: UOp
    cache_tag: bytes
    diagnostics: tuple[Diagnostic, ...] = ()

```

The hook logic remains:

```

lowered = try_lower_graph(function)
if lowered is None:
    cache_key = function.key
    graph = get_kernel_graph(function)
else:
    lowerer_name, result = lowered
    cache_key = sha256(function.key + lowerer_name + result.cache_tag).digest()
    graph = result.graph

linear = schedule_cache.get(cache_key)
if linear is None:
    type_verify(function, spec_tensor)      # when SPEC is enabled
    linear = create_schedule(graph)
    schedule_cache[cache_key] = linear
return linear

```

5.2 TT lowerer surface

```

@dataclass(frozen=True)
class TTCapabilities:
    target: str
    topology_hash: bytes
    l1_bytes: int
    data_buffer_base: int
    param_slots: int
    cb_ids: int | UnknownLimit
    role_text_bytes: Mapping[KernelRole, int]
    role_local_bytes: Mapping[KernelRole, int]
    sfpu_lregs: int | UnknownLimit
    replay_entries: int | UnknownLimit
    dst_tiles: int | UnknownLimit
    noc_max_burst: int
    supported_dtypes: frozenset[DType]
    verified: frozenset[str]

@dataclass(frozen=True)
class TTLoweringOptions:
    numerical_mode: Literal["correctness", "hifi2", "lofi"]
    layout_policy: Literal["staged_row_major", "persistent_tiled"]

```



```

max_cores: int
allow_recompute: bool
diagnostics: Literal["error", "collect"]

class TTGraphLowerer:
    def __init__(self, caps: TTCapabilities, options: TTLoweringOptions): ...
    def __call__(self, request: GraphLoweringRequest) -> BackendGraph | None:
        roots = classify_roots(request)
        if not self.claims(roots): return None
        graph = analyze_shaped_graph(request.function, roots)
        recipes = match_recipes(graph)
        candidates = enumerate_regions(graph, recipes)
        plan = partition_deterministically(candidates, self.caps, self.options)
        artifacts = tuple(emit_region(region, self.caps) for region in plan.regions)
        return BackendGraph(build_tinygrad_call_graph(plan, artifacts), plan.cache_tag)

```

claims must be strict. A TT-prefixed device leaf is insufficient if another root performs unrelated compute on another device. Diagnostics should look like:

```

TT graph lowerer declined root 2:
  effect: compute
  target device: CPU
  value devices: {CPU, TT}
  crossing: non-COPY arithmetic consumes both CPU and TT buffers
  supported first-version mixed roots: TT compute/mutation plus ingress/egress COPY only

```

5.3 Recipe and candidate-region API

```

@dataclass(frozen=True)
class RecipeMatch:
    recipe_id: str
    roots: tuple[UOp, ...]
    covered: frozenset[UOp]
    inputs: tuple[ValueInfo, ...]
    outputs: tuple[ValueInfo, ...]
    mutations: tuple[Mutation, ...]
    algorithms: tuple[AlgorithmVariant, ...]

@dataclass(frozen=True)
class AlgorithmVariant:
    name: str
    requirements: ResourceFormula
    layout_constraints: tuple[LayoutConstraint, ...]
    numerical_contract: NumericalContract
    emitter: KernelTemplateFactory

@dataclass(frozen=True)
class RegionCandidate:
    matches: tuple[RecipeMatch, ...]
    external_inputs: tuple[ValueInfo, ...]

```

```

external_outputs: tuple[ValueInfo, ...]
mutations: tuple[Mutation, ...]
phases: tuple[Phase, ...]
partition: CorePartition
resources: RegionResources
signature: bytes

```

Matching should report both successful recipes and the first unsupported subexpression. It should never silently fall back from an exact semantic pattern to a numerically different algorithm.

5.4 Immutable bundle artifact and runtime ABI

The current rewrite Program is close to an executable carrier but does not include enough semantic ABI metadata. **[P]** Add a compiler-owned artifact above it:

```

@dataclass(frozen=True)
class TensorPort:
    slot: int
    name: str
    access: Literal["read", "write", "readwrite"]
    dtype: DType
    logical_shape: tuple[sint, ...]
    physical_shape: tuple[sint, ...]
    layout: Layout
    aliases: tuple[int, ...] = ()

@dataclass(frozen=True)
class ScalarPort:
    slot: int
    name: str
    dtype: DType
    bounds: tuple[int, int] | None

@dataclass(frozen=True)
class RoleImage:
    core: Core
    role: KernelRole
    state: Literal["active", "empty_return"]
    code: bytes
    code_hash: bytes

@dataclass(frozen=True)
class TTBundleArtifact:
    abi_version: int
    target: str
    capability_hash: bytes
    source_fingerprint: bytes
    cores: tuple[Core, ...]
    partition: CorePartition
    tensor_ports: tuple[TensorPort, ...]
    scalar_ports: tuple[ScalarPort, ...]

```

```

role_images: tuple[RoleImage, ...]
cbs: tuple[CBAallocation, ...]
barriers: tuple[BarrierAllocation, ...]
phases: tuple[PhaseManifest, ...]
mutations: tuple[Mutation, ...]
resources: RegionResources
diagnostics: tuple[Diagnostic, ...]

@property
def key(self) -> bytes: ...      # hash of all immutable fields and code
def to_program(self) -> Program: ...

```

Compilation returns immutable bytes and metadata. Runtime binding is separate:

```

class TTRuntimeProgram:
    def __init__(self, device: TTDevice, artifact: TTBundledArtifact): ...

    def __call__(
        self, *buffers: TTBufferHandle,
        global_size=None, local_size=None,
        vals: tuple[int, ...] = (), wait=False, timeout=None,
    ) -> float | None:
        bound = self.validate_and_bind(buffers, vals)
        self.device.ensure_uploaded(self.artifact)
        commands = self.build_binding_and_launch_commands(bound)
        return self.device.submit(commands, wait=wait, timeout=timeout)

```

`validate_and_bind` must compare port identity, dtype, logical shape, physical shape, layout, access/mutation rights, and address width. The current rewrite's `Buffer` records logical and padded shape but no layout/strides/alias contract (blackhole-py-rewrite/program.py:22-41), so layout must be added before persistent tiled buffers are considered ABI-compatible.

5.5 Encoding the artifact in an ordinary Tinygrad PROGRAM

Tinygrad's progressive spec permits a PROGRAM with semantic SINK, lowered LINEAR, SOURCE, and BINARY sources (tinygrad/tinygrad/uop/spec.py:176-183; tinygrad/tinygrad/codegen/__init__.py:223-257). **[P]** Use those existing fields rather than a TT Op:

```

semantic_sink = region.semantic_sink
manifest_json = stable_json(artifact.without_code())
code_blob = serialize_role_images(artifact.role_images)

program = UOp(
    Ops.PROGRAM,
    dtypes.void,
    src=(
        semantic_sink,
        UOp(Ops.LINEAR, src=()),          # already complete; no scalar lowering
        UOp(Ops.SOURCE, dtypes.void, arg=manifest_json),
        UOp(Ops.BINARY, dtypes.void, arg=code_blob),
    ),
)

```

```

arg=ProgramInfo(
    name=stable_name(region),
    globals=tuple(port.slot for port in artifact.tensor_ports),
    outs=tuple(port.slot for port in artifact.tensor_ports if port.access != "read"),
    ins=tuple(port.slot for port in artifact.tensor_ports if port.access != "write"),
    vars=tuple(symbolic_uops),
    aux=(artifact.abi_version, artifact.key, artifact.resources),
),
)

call = program.call(*ordered_param_uops)
roots = tuple(param.after(call) for param in written_or_mutated_params)
return UOp.sink(*roots)

```

The exact `ProgramInfo` constructor must follow the captured fields rather than this abbreviated sketch, but the contract is supported: `SINK`, `PROGRAM`, `LINEAR`, `COPY`, `SLICE`, and `CUSTOM` are opaque call bodies (tinygrad/tinygrad/uop/ops.py:1062-1071), and `ProgramInfo.from_sink` already derives ordered globals/outputs/inputs and variables (tinygrad/tinygrad/uop/ops.py:1114-1135).

5.6 TTK-to-bundle construction API

```

class TTBundleBuilder:
    def __init__(
        self, cores: tuple[Core, ...],
        tensor_ports: tuple[TensorPort, ...],
        scalar_ports: tuple[ScalarPort, ...],
        caps: TTCapabilities,
    ): ...

    def cb(self, spec: CBSpec, lifetime: PhaseInterval) -> CBEndpoint: ...
    def barrier(self, spec: BarrierSpec) -> PhaseToken: ...
    def emit(self, role: KernelRole, template: KernelTemplate) -> None: ...
    def verify(self) -> RegionResources: ...
    def lower(self) -> TTBundleArtifact: ...

```

`lower` creates a fresh `KernelBuilder` per core/role, invokes each template with the deterministic tile span and allocated endpoints, requires every role to terminate, emits an explicit return image for inactive roles, measures exact bytes/local allocations/replay use, and performs a final verifier pass. It then wraps the current `KernelBundle/Program` or replaces them with equivalent immutable dataclasses.

6. Fusion legality, resource accounting, and deterministic partitioning

6.1 What the five central probes establish

[E] Corpus regeneration. In the full documented probe order, all renderer-neutral stages and call ABI regenerated byte-for-byte for `matmul_epilogue`, `rmsnorm`, `sdpa_gqa_decode`, `kv_cache_update_symbolic`, and `llama32_1b_block_decode` (`research-output/uop_renderer_neutral_regeneration.out:1-6`). Late Python renderer stages were deliberately excluded from that determinism claim because process-global identifiers can perturb their text.

[I] `matmul_epilogue` is one generic call. The AST contains a K=64 `REDUCE` over a multiply, then bias add and a `WHERE` implementing `ReLU`, stored directly to 128 outputs (`uop-dumps/current/matmul_epilogue/50_kernel_00_base_ast.txt:4-34`). Generic Tinygrad can already fuse a conventional epilogue; the TT planner should preserve and extend this behavior, not assume one operator per program.

[I] `rmsnorm` is two calls. Call 0 materializes one reciprocal square-root scalar from the sum of squares, $1/2048$, and epsilon (`uop-dumps/current/rmsnorm/50_kernel_00_base_ast.txt:4-22`). Call 1 multiplies input, scalar, and gamma across 2048 outputs (`uop-dumps/current/rmsnorm/50_kernel_01_base_ast.txt:4-22`). A TT program can plausibly implement those as two phases without a global scalar buffer.

[I] standalone decode GQA SDPA is four calls. The first computes 32×18 QK scores with head mapping `q_head // 4` and scale 0.125 (`uop-dumps/current/sdpa_gqa_decode/50_kernel_00_base_ast.txt:4-43`). Subsequent calls compute the stable row maximum, exponent-sum reciprocal, and the probability-weighted V reduction (`uop-dumps/current/sdpa_gqa_decode/50_kernel_01_base_ast.txt:4-21`, `uop-dumps/current/sdpa_gqa_decode/50_kernel_02_base_ast.txt:4-31`, `uop-dumps/current/sdpa_gqa_decode/50_kernel_03_base_ast.txt:4-54`). The final call recomputes/consumes normalized weights rather than requiring a separately visible probability tensor in this standalone graph.

[I] symbolic KV update is one call. Its single AST stores into a 131072-element cache and carries `start_pos` as a bounded symbolic `PARAM`, with guarded K/V selection (`uop-dumps/current/kv_cache_update_symbolic/50_kernel_00_base_ast.txt:4-47`; `research-output/corpus_inventory.out:35-39`). The mutation is a state/effect boundary, not necessarily a separate program boundary.

6.2 Why the Llama block becomes 14 calls

The final call inventory is reproducible and recorded at `research-output/corpus_inventory.out:41-97`. The semantic labels below are **[N]** derived by matching each AST's shapes, parameters, reductions, exponentials, and buffer dependencies back to the shaped block graph.

Call	Output	Current work	Why it is a boundary now	Plausible TT grouping
0	1	Input RMSNorm reciprocal scale	Generic reduction result materialized for multiple projection consumers.	Phase-local scalar inside QKV program.
1	2048	Normalized Q projection	Separate <code>matmul</code> output feeds Q RoPE/attention. Normalization multiply is already fused.	Group with calls 0, 2, 3, and possibly 4.
2	512	Normalized K projection	Separate <code>matmul</code> output feeds K RoPE/cache.	Group with input norm/QKV/RoPE/cache.

Call	Output	Current work	Why it is a boundary now	Plausible TT grouping
3	131072 cache state	V projection plus K-side RoPE/KV cache update	Persistent mutation is represented by STORE → AFTER; V projection and K update are already fused into the state-producing call.	Same bundle as QKV with an explicit write phase, or a dedicated QKV/RoPE/KV program.
4	4096	Q RoPE plus QK scores over max context	Score matrix becomes a softmax input.	Attention bundle phase 1.
5	32	Row maximum	Stable softmax reduction materialization.	Attention bundle phase-local max.
6	32	Reciprocal exponent sum	Second stable softmax reduction materialization.	Attention bundle phase-local denominator/online state.
7	4096	Normalized probabilities	Block graph materializes probabilities before V reduction.	Eliminate with online softmax or keep in L1/CB within attention bundle.
8	2048	Probability × cached V reduction	End of attention core.	Attention bundle final phase.
9	2048	O projection plus first residual add	Matmul epilogue is already fused; output feeds second norm.	Group with residual and possibly second RMSNorm phase.
10	1	Second RMSNorm reciprocal scale	Generic reduction scalar materialized for MLP projections.	Phase-local scalar in MLP or O/residual/norm bundle.
11	8192	Gate projection plus SiLU	The model explicitly calls <code>contiguous()</code> on the gate result.	Fuse with call 12 as gate/up/SwiGLU.
12	8192	Up projection multiplied by materialized gate	Consumes call 11 buffer.	Same gate/up/SwiGLU program.
13	2048	Down projection plus final residual	Final externally visible block output.	MLP output program; may remain a natural boundary.

The block's shaped graph contains two different kinds of forced state:

- The cache update is a real mutation: STORE followed by AFTER, then a slice of the updated cache (`uop-dumps/current/llama32_1b_block_decode/00_shaped_sink.jsonl:159-162`). This must remain an ordering edge, but it can be a phase inside the same TT program if subsequent reads observe the update correctly.
- The FFN gate has an explicit CONTIGUOUS (`uop-dumps/current/llama32_1b_block_decode/00_shaped_sink.jsonl:265-268`). Tinygrad's model source labels this as a temporary workaround (`tinygrad/tinygrad/llm/model.py:100-121`). Removing it in the host-only probe reduced 14 calls to 13; removing the outer final CONTIGUOUS alone did not (`research-output/fusion_boundary_experiment.out:1-12`).

Callification preserves the cache STORE/AFTER and converts output placement into explicit STORE/AFTER; it retains the gate materialization as the one internal contiguous boundary (`uop-dumps/current/llama32_1b_block_decode/11_callified_inner_sink.jsonl:162-163`, `uop-dumps/current/llama32_1b_block_decode/11_callified_inner_sink.jsonl:268`, `uop-dumps/current/llama32_1b_block_decode/11_callified_inner_sink.jsonl:295-309`). Thus the 14-call count is explained by current generic reduction/softmax materialization, explicit state/output semantics, and one workaround—not a blanket lack of fusion.

6.3 Candidate block partitions

[P] The compiler should enumerate, verify, and compare at least these candidate regions:

1. Input RMSNorm + Q/K/V projections + Q/K RoPE + KV update.
2. GQA score/max/sum/value attention, preferably online or with phase-local score/probability storage.
3. O projection + first residual + second RMSNorm.
4. Gate and up projections + SiLU/product (SwiGLU).
5. Down projection + final residual.

That is the five-program optimistic decomposition. Legal alternatives split QKV, RoPE/KV, attention, or norm when code/L1/Dst/core/synchronization constraints require it, producing six to eight or more programs. No exact count should be hard-coded. The old hand path itself invokes many separate high-level programs (`blackhole-py/examples/llama3/block.py:39-149`), while its fused SwiGLU demonstrates that at least one current generic materialization can be removed in a purpose-built bundle (`blackhole-py/examples/llama3/swiglu.py:1-11`).

6.4 Hard semantic boundaries versus optional materializations

A region boundary is mandatory when any of the following holds:

- An externally visible output must exist before host or another device observes it.
- A persistent mutation cannot be ordered safely inside one phase graph, or an alias may observe old/new state ambiguously.
- A cross-device edge is not an ordinary supported copy.
- The required movement/indexing/layout conversion has no in-bundle implementation.
- Numerical semantics require an intermediate precision/rounding point that fusion would change beyond the selected contract.
- Different roots require incompatible compile-time symbolic maxima or core topology.
- Any verified resource limit is exceeded.

A `CONTIGUOUS`, reduction scalar, score matrix, softmax max/denominator, probability matrix, or projection output is optional **only** if the fused algorithm preserves semantics and passes resources. `AFTER` is an ordering/state relation; it is not automatically a program boundary.

6.5 Deterministic resource model

[I] **Known current rewrite limits:**

- L1 size `0x180000`; data-buffer region begins at `0x37000` (`blackhole-py-rewrite/fw/consts.py:3-16`).
- Parameter table starts at `0x4240`, has size `0xEC0`, and therefore 944 32-bit slots (`blackhole-py-rewrite/fw/consts.py:6-8`).
- Each role has a fixed `0x9F00` worker-text partition (`blackhole-py-rewrite/fw/consts.py:9-16`).
- Role-local RAM ranges are explicit (`blackhole-py-rewrite/asm.py:16-20`).
- `kernelBundle` currently permits 32 CB IDs and verifies their backing range in L1 (`blackhole-py-rewrite/program.py:183-190`).
- SFPU state exposes eight local registers (`blackhole-py-rewrite/ttk/sfpu.py:63-111`).
- The Tensix replay allocator has 32 entries (`blackhole-py-rewrite/ttk/tensix.py:147-180`).
- One NoC command is limited to 16 KiB, and multicast explicitly chunks larger transfers (`blackhole-py-rewrite/ttk/noc.py:6-8`, `blackhole-py-rewrite/ttk/noc.py:336-355`).

[U] **Unknown or not authoritative in this bundle:** exact hardware CB namespace beyond the rewrite's software limit, Dst tile capacity by format/mode, complete reset state, exact SFPU operation/precision behavior, maximum outstanding NoC transactions, collective/topology costs, and any fixed external tensor-port hardware limit. The design

document's reference to TT-Metal exposing 64 CBs depends on omitted sources (blackhole-py-rewrite/tenstorrent-lowering.md:639-649; metadata/EXCLUSIONS.md:23-31). The compiler capability table must mark these as Unknown, disable algorithms that require them, and fail with a named evidence gap rather than guessing.

A candidate's resource record should be exact where emission can measure it and symbolic/bounded where shape drives it:

```
@dataclass(frozen=True)
class RegionResources:
    external_tensor_ports: tuple[TensorPortUse, ...]
    scalar_param_slots: int
    cb_allocations: tuple[CBAallocation, ...]
    peak_cb_l1_bytes: int
    scratch_allocations: tuple[ScratchAllocation, ...]
    role_code_bytes: Mapping[KernelRole, int]
    role_local_bytes: Mapping[KernelRole, int]
    dst: DstRequirement
    sfpu_live_lregs: int
    replay_entries: int
    noc_streams: tuple[NocStreamRequirement, ...]
    noc_chunks: int
    core_partition: CorePartition
    phases: tuple[PhaseRequirement, ...]
    layout_conversions: tuple[LayoutConversion, ...]
    external_dram_read_bytes: sint
    external_dram_write_bytes: sint
    recomputed_ops: sint
    mutations: tuple[Mutation, ...]
```

CB allocation is an interval problem over phase/lifetime intervals. Assign stable CB IDs by sorting (`first_phase`, `last_phase`, `semantic_name`, `producer_role`, `consumer_role`) and first-fit coloring into the verified namespace. Assign backing L1 with aligned first-fit-decreasing by size, using a deterministic name/signature tie-break. Verify wrap-safe occupancy from producer/consumer tile counts and phase overlap; page depth is not merely a byte total.

Per-role code and local memory should be measured by a dry-run emission through fresh builders—the current builder already exposes exact bytes and has fixed local allocator ranges. Formula estimates are useful for candidate pruning, but final legality is exact emitted size.

6.6 Deterministic core partition

For a one-dimensional tile domain of N tiles and an ordered available core list:

```
C = min(max_cores, N)
core i owns [ floor(i*N/C), floor((i+1)*N/C) )
```

This is the same balanced contiguous split used by the historical fused SwiGLU (blackhole-py/examples/llama3/swiglu.py:283-292). Empty cores are not selected. The core order is a canonical row-major list after harvesting/topology filtering; its hash is part of capabilities.

For a two-dimensional domain, enumerate legal (`rows`, `cols`) rectangles up to the available grid. For each rectangle, deterministically block or cyclically map the logical tile grid according to the recipe's supported distributions, compute:


```
(max_tiles/core,  
  load_imbalance,  
  halo/collective_bytes,  
  DRAM_bank_conflict_estimate,  
  idle_cores,  
  rows,  
  cols,  
  mapping_signature)
```

and choose lexicographically. Symbolic dimensions compile against declared maxima; launch values provide actual bounds and each core's valid span/mask.

6.7 Fusion objective and deterministic splitting

For each legal complete partition, minimize lexicographically:

```
(  
  number_of_programs,  
  externally_materialized_padded_bytes,  
  layout_conversion_bytes,  
  NoC_bytes,  
  peak_L1_bytes,  
  maximum_role_code_bytes,  
  recomputed_operation_cost,  
  stable_plan_signature,  
)
```

This intentionally prioritizes program count only among legal plans, then real materialization/conversion traffic. It does not fuse merely because an operation count is smaller.

When a candidate region fails, enumerate all legal cut sets that separate it into already matched subregions. Score each cut by:

```
(materialized_padded_bytes,  
  layout_conversions,  
  number_of_cut_edges,  
  mutation_crossings,  
  earliest_topological_cut_index,  
  stable_value_ids)
```

Choose the minimum, recursively verify both sides, and emit a diagnostic trace:

```
region llama.attention.qk_to_pv rejected as one program  
  limiting resource: peak CB backing  
  required: 425,984 bytes  
  verified available: 397,312 bytes  
  largest live allocations:  
    scores: 262,144 bytes, phases 1..3  
    q_tiles: 65,536 bytes, phases 0..1  
    value_tiles: 65,536 bytes, phases 3..4
```

```
chosen cut after softmax denominator
  added external materialization: 4,096 bytes
  alternative cut after scores: 262,144 bytes
```

For the first implementation, a deterministic maximal-fusion dynamic program over topologically contiguous recipe groups is adequate. It should enumerate every interval, retain Pareto-optimal legal variants by output state/layout, and solve the minimum lexicographic path. More general DAG partitioning can follow; an unspecified greedy heuristic should not.

7. TTK design, ownership, and synchronization rules

7.1 What current rewrite TTK actually provides

The rewrite is already more than a raw instruction encoder, but less than the public compiler layer described in `TTK.md`.

- **[I] Common/RISC support:** scoped registers, loads/stores, branches, calls, labels, local allocation, and loading an address from the program parameter table (`blackhole-py-rewrite/asm.py:206-239`; `blackhole-py-rewrite/ttk/common.py:155-169`).
- **[I] NoC:** initialization/state, reads, writes, posted/acknowledged batches, streams, multicast chunking, atomics, and completion tickets (`blackhole-py-rewrite/ttk/noc.py:277-366`). The per-command burst cap is 16 KiB (`blackhole-py-rewrite/ttk/noc.py:6-8`).
- **[I] CB:** configuration reset and local-interface plus producer/consumer reserve/wait/push/pop operations (`blackhole-py-rewrite/ttk/cb.py:4-29`, `blackhole-py-rewrite/ttk/cb.py:45-112`).
- **[I] Unpack/math/pack:** one-source unpack to SrcA, SrcA-to-Dst copy plus SFPU, and one-tile pack to a CB (`blackhole-py-rewrite/ttk/unpack.py:12-50`, `blackhole-py-rewrite/ttk/math.py:6-44`, `blackhole-py-rewrite/ttk/pack.py:5-48`).
- **[I] SFPU:** limited LReg/replay and add-immediate functionality (`blackhole-py-rewrite/ttk/sfpu.py:63-153`).
- **[I] Tensix state/replay:** 32 replay entries, context/MOP/thread/SFPU tracking, and suppression of redundant writes (`blackhole-py-rewrite/ttk/tensix.py:147-215`, `blackhole-py-rewrite/ttk/tensix.py:247-285`).
- **[I] Sync:** polling barrier support (`blackhole-py-rewrite/ttk/sync.py:1-8`).
- **[I] DRAM topology helper only:** harvested-bank endpoint coordinates, but no tensor page/tile address generator (`blackhole-py-rewrite/ttk/dram.py:1-21`).

What is absent is the durable compiler-facing layer: typed tensor ports, declarative operation templates, exact engine preconditions/effects, multi-phase ownership, resource formulas, reduction/matmul/address-generation recipes, safe control-flow state merge, and inspectable artifacts.

7.2 Public surface: templates over an imperative escape hatch

[P] Keep `KernelBuilder` public as the low-level escape hatch, but make compiler-generated kernels use declarative templates with explicit requirements and effects.

```
Role = Literal["brisc", "ncrisc", "trisc0", "trisc1", "trisc2"]

@dataclass(frozen=True)
class TileSpan:
    first: sint
    last: sint
    valid_tiles: sint
    masks: tuple[TileMask, ...]

@dataclass(frozen=True)
class RoleRequirements:
    code_estimate: int
    local_bytes: int
    cb_endpoints: tuple[CBRequirement, ...]
    dst: DstRequirement
    sfpu_lregs: int
```

```

replay_entries: int
noc_streams: tuple[NocStreamRequirement, ...]
config_reads: frozenset[ConfigResource]
config_writes: frozenset[ConfigResource]
barriers: tuple[BarrierRequirement, ...]

@dataclass(frozen=True)
class RoleEffects:
    cb_produced: tuple[TileCount, ...]
    cb_consumed: tuple[TileCount, ...]
    dst_state: DstState
    engine_state: ExactTensixState
    noc_tickets: tuple[NocTicket, ...]
    config_writes: frozenset[ConfigResource]

class RoleContext(Generic[RoleT]):
    asm: KernelBuilder
    state: ExactTensixState
    resources: ResourceRecorder
    ports: PortAccessor

    def param(self, port: TensorPort | ScalarPort) -> RuntimeValue: ...
    def cb(self, endpoint: CBEndpoint) -> TypedCB: ...
    def barrier(self, token: PhaseToken) -> TypedBarrier: ...
    def noc(self, index: Literal[0, 1]) -> Noc: ...
    def scratch(self, spec: ScratchSpec) -> ScratchRef: ...

class KernelTemplate(Protocol):
    template_id: str
    role: Role
    def requirements(self, spec: TemplateSpec) -> RoleRequirements: ...
    def emit(self, ctx: RoleContext, spec: TemplateSpec, span: TileSpan) -> RoleEffects: ...

```

Templates should cover stable architectural sequences rather than model names: DRAM tiled reader/writer, CB relay, unpack A/B, BF16/FP32 matmul blocks, row reduction, scalar broadcast, SFPU expression, pack, multicast reduction, barrier, and page-address generators. Higher recipes such as RMSNorm and attention compose these primitives into phases.

The imperative path remains:

```
bundle.emit("trisc1", RawKernelTemplate(lambda k: ...))
```

but it must declare conservative requirements/effects or be rejected from automatic fusion. This permits bring-up and hand tuning without making uninspectable assembler callbacks the default compiler ABI.

7.3 Exact state, not optimistic zero defaults

[I] The current state model is unsafe as a general compiler proof because an unobserved CFG register is read as zero. TensixState.cfg and set_cfg use .get(..., 0) (blackhole-py-rewrite/ttk/tensix.py:193-201). That can suppress a write if firmware or a predecessor left a nonzero value.

[P] Represent every shadow as Known(raw_bits) or Unknown:

```

@dataclass(frozen=True)
class Known:
    raw: int

StateValue = Known | Literal[Unknown]

@dataclass(frozen=True)
class ExactTensixState:
    cfg: Mapping[tuple[Pipe, Context, Cfg], StateValue]
    thread_cfg: Mapping[tuple[Pipe, int], StateValue]
    mop: Mapping[Pipe, MopState | Unknown]
    sfpu: SfpuState | Unknown
    dst: DstState | Unknown
    unpack: UnpackState | Unknown
    pack: PackState | Unknown

```

Rules:

1. Fresh state begins from a versioned, evidence-backed firmware handoff contract. Anything not established by current firmware/source or a template preamble is `Unknown`.
2. A write may be elided only when the previous value is the identical `Known` raw value.
3. A read-modify-write on `Unknown` is illegal unless the template first establishes the complete register.
4. Each template declares required known state and resulting state; the compiler either inserts a state-establishing preamble or rejects the transition.
5. Every core and role starts with a distinct state object. Shared hardware resources are modeled separately rather than accidentally shared through Python objects.

7.4 Safe merge across control flow

At a CFG join, merge each state component independently:

```

def merge(values: Iterable[StateValue]) -> StateValue:
    vals = tuple(values)
    return vals[0] if vals and all(v == vals[0] for v in vals[1:]) else Unknown

```

MOP/replay allocations, CB occupancy, outstanding NoC tickets, and Dst ownership require structured merges, not just register-bit equality. For example, a replay slot is reusable after a branch only if every predecessor frees the same range; otherwise the joined allocator state is conservative/unknown and the template must synchronize/reset or avoid the branch.

The first compiler should avoid data-dependent kernel CFG entirely except bounded loops with statically identical state at backedges. Symbolic launch values can determine loop trip counts and masks without creating arbitrary divergent state.

7.5 Ownership rules for five concurrent RISCs

[P] Make ownership explicit and reject undeclared concurrent writers. The initial policy should match the demonstrated `add1` pipeline ([blackhole-py-rewrite/examples/add1.py:23-67](#)):

Resource	Initial owner(s)	Rule
DRAM/NoC reads	BRISC	BRISC produces input CB pages; a recipe may assign a separate NoC instance to NCRISC only through an explicit capability/phase rule.
DRAM/NoC writes	NCRISC	NCRISC consumes output CB pages and acknowledges writes.
Unpack configuration and SrcA/SrcB loading	TRISC0	Only TRISC0 mutates unpack-owned registers during an active phase.
Math, SFPU, Dst production	TRISC1	Only TRISC1 owns Dst compute state and SFPU state.
Pack configuration and Dst consumption	TRISC2	TRISC2 acquires/packs/releases Dst under a declared producer-consumer protocol.
CB producer counters	Declared producer role	Exactly one producer unless a multi-producer protocol is explicitly modeled.
CB consumer counters	Declared consumer role	Exactly one consumer in the first implementation.
Shared Tensix CFG/registers	One owner per phase	A phase transition barrier is required before ownership changes.
Barrier words/semaphores	Compiler-assigned	Parties, initialization, and one-shot/reuse generation are explicit.

A PhaseManifest should state ownership and synchronization:

```
@dataclass(frozen=True)
class PhaseManifest:
    index: int
    name: str
    active_roles: frozenset[Role]
    config_owners: Mapping[ConfigResource, Role]
    cb_edges: tuple[CBFlow, ...]
    entry_barriers: tuple[PhaseToken, ...]
    exit_barriers: tuple[PhaseToken, ...]
    must_flush_noc: tuple[NocTicketSet, ...]
    dst_transition: DstTransition
```

If TRISC0 and TRISC1 both write one shared config resource in the same phase with no protocol, compilation fails. If ownership changes at a phase boundary, all prior users must reach the boundary, outstanding engine/NoC work must be complete, and the new owner must establish any state that is not proven preserved.

7.6 CB correctness and occupancy

Typed CB endpoints should bind dtype/page size, depth, producer, consumer, and lifetime:

```
@dataclass(frozen=True)
class CBSpec:
    name: str
    dtype: DType
    page_bytes: int
    depth: int
    producer: Role
    consumer: Role
```

```
lifetime: PhaseInterval
initial_tiles: int = 0
```

The verifier checks:

- exactly matched production/consumption counts for every path;
- $0 \leq \text{produced} - \text{consumed} \leq \text{depth}$ at every phase/loop prefix;
- reserve occurs before write, push after write completion, wait before read, pop after use;
- page size matches unpack/pack/tile format;
- backing ranges are aligned, disjoint while live, and inside the data-buffer region;
- reused CB IDs/backing memory have non-overlapping lifetimes and reset counters before reuse.

This is stronger than merely counting CB IDs or bytes.

7.7 Architectural expression helpers

The public SFPU/math layer should accept typed architectural expressions, not arbitrary Python ASTs:

```
x = SrcA.tile(0, dtype=BF16)
g = SrcB.tile(0, dtype=BF16)
s = Local.scalar("inv_rms", dtype=F32)
out = (x.to(F32) * s * g.to(F32)).to(BF16, rounding=RNE)
ctx.sfpu.emit(out, dst=Dst.tile(0))
```

Expression lowering must expose register pressure, supported op/format combinations, approximation mode, and rounding. Unsupported expressions fail during recipe verification. The compiler may fuse a chain only if the resulting expression fits LRegs/replay/Dst and preserves the selected numerical contract.

7.8 What TTK should not abstract

TTK should **not** own:

- Tinygrad Tensor semantics, graph matching, alias analysis, or AFTER ordering;
- graph-level fusion/partition objectives;
- global Tinygrad buffer lifetime/memory planning;
- hidden firmware/CQ behavior;
- opaque “automatic synchronization” that obscures barriers or tickets;
- layout conversions without explicit bytes/cost/effects;
- hardware limits unsupported by evidence;
- a model-specific “Llama kernel” API.

It should emit inspectable role code and exact state/resources from explicit templates. The compiler above it owns why templates are composed.

7.9 What to port from blackhole-py

[H] Port selectively: proven low-level sequences for matmul, RMSNorm, RoPE, KV access, attention score streaming, softmax, residual, SwiGLU, per-core RTAs, and ordered CQ batching. The old source demonstrates deterministic contiguous sharding and complete five-role programs, for example fused SwiGLU (blackhole-py/examples/llama3/swiglu.py:283-357) and attention score streaming (blackhole-py/examples/llama3/attn.py:1550-1640).

[P] Redesign: dynamic mixin-style firmware builders, unconstrained per-kernel RTA lists, implicit shared-register ownership, hand-maintained CB IDs, model-specific address arithmetic, and mutable `Program` objects. Convert useful sequences into typed templates with requirements/effects and golden assembly/resource tests.

[P] Delete or quarantine: duplicate stale helpers, APIs whose documented state differs from code, and any helper whose correctness depends on omitted TT-LLK behavior without a local test or explicit evidence marker.

8. Layout, memory, and runtime strategy

8.1 The current mismatch

[I] Tinygrad's generic buffer abstraction is flattened. BufferSpec contains cache/host/external-pointer flags, not logical shape or layout (tinygrad/tinygrad/device.py:77-85). A Buffer stores device, element count, dtype, byte offset, and base/view identity; allocator allocation receives only byte size/options (tinygrad/tinygrad/device.py:99-160, tinygrad/tinygrad/device.py:223-248). Views require allocator _offset support (tinygrad/tinygrad/device.py:132-156, tinygrad/tinygrad/device.py:217-219). Generic memory planning only includes devices whose allocators implement _offset (tinygrad/tinygrad/schedule/memory.py:13-16).

[I] The rewrite already distinguishes logical and padded shape, but not layout. Its Buffer has shape and padded_shape; physical size/pages derive from padded elements and dtype (blackhole-py-rewrite/program.py:22-41). Host conversion demands that the last two padded dimensions be multiples of 32 and performs a fixed 32×32 tile/face permutation (blackhole-py-rewrite/device.py:78-92). It does not encode row-major versus tiled layout in the buffer ABI, nor arbitrary strides/subviews.

Therefore, silently allocating a Tinygrad flat byte buffer and treating it as a persistent tiled tensor is incorrect. Logical shape cannot be reconstructed reliably from nbytes, and generic byte offsets are not generally valid tile subviews.

8.2 Correctness-first initial design

[P] Start with Tinygrad-visible row-major logical buffers and private runtime tiled staging.

1. Tinygrad allocates a normal opaque TT buffer handle with logical element count/dtype.
2. The handle carries runtime side metadata: logical shape, padded shape, row-major host layout, and a lazily allocated tiled DRAM backing object.
3. Host-to-device copy pads logical data, then uses the rewrite's NumPy tile/face transform and queues a DRAM transfer (blackhole-py-rewrite/device.py:50-107). Device-to-host performs the inverse and removes padding.
4. A compiled bundle declares tiled ports. Before launch, runtime ensures each input's tiled backing is current. After a write, row-major host state is marked stale; copyout utilizes on demand.
5. Intermediate values within one TT program remain in CB/L1/Dst and never become Tinygrad buffers.
6. Inter-program TT outputs initially materialize in tiled DRAM through runtime-owned handles with explicit metadata. Generic Tinygrad byte-offset memory planning is disabled for these allocations until tile-aware _offset exists.

This design is slower but coherent. It permits add1, elementwise, reduction, matmul, and one-block correctness before modifying Tinygrad's allocator API.

8.3 Persistent tiled destination

[P] After correctness, add one generic layout-aware allocation hook rather than TT conditionals throughout Tensor code. It must receive logical shape and dtype before flattening and return a logical buffer view over a physical allocation descriptor:

```
@dataclass(frozen=True)
class BufferLayout:
    logical_shape: tuple[sint, ...]
    physical_shape: tuple[sint, ...]
    storage_dtype: DType
    encoding: Literal["row_major", "tt_tile_faces"]
```

```

tile: tuple[int, int] | None
strides_or_tile_map: TileMap
valid_mask: TileMask

class LayoutAllocator(Allocator):
    def alloc_layout(self, layout: BufferLayout, options: BufferSpec): ...
    def offset_layout(self, base, view: BufferLayout, logical_offset: ViewSpec): ...

```

Only tile-aligned, representable views may use `_offset`; arbitrary Tinygrad slices either remain logical views consumed by the compiler, materialize through a copy/retile program, or fail with a diagnostic. Once this exists, generic lifetime planning can suballocate physical tile ranges safely.

8.4 Logical shape, padding, partial tiles, and movement

For each value, the compiler carries both shapes. A 32×32 tile is the storage/computation unit, but padding is semantic only when masked:

- Elementwise output stores mask invalid rows/columns.
- Reductions inject the operation identity in invalid lanes: zero for sum, `-inf` for max, one for product where supported.
- Broadcasts can reuse one tile/row/column without materialization if the engine recipe supports the access.
- Logical reshape is free only when it preserves the same linear/tile map.
- Permute may be a view for tile-grid dimension swaps supported by address generation; intra-tile permutations need an engine or conversion.
- Pad/shrink/flip become guarded address/mask operations when affine; otherwise they require conversion.
- Tensor-indexed gather needs a dedicated indexed NoC/address recipe. It is not approximated by affine strides.

Each conversion is an explicit phase or separate program with accounted bytes.

8.5 DRAM addressing and bank distribution

[I] Current rewrite DRAM allocation stripes pages across seven banks conceptually, and transfer programs distribute pages over selected cores (blackhole-py-rewrite/program.py:48-63, blackhole-py-rewrite/device.py:50-76). ttk/dram.py only computes bank endpoint coordinates (blackhole-py-rewrite/ttk/dram.py:1-21).

[P] Add a typed address generator:

```

@dataclass(frozen=True)
class DramTensorLayout:
    base: RuntimeAddress
    bank_count: int
    page_bytes: int
    tile_shape: tuple[int, int]
    grid_shape: tuple[int, ...]
    bank_policy: Literal["round_robin_pages", "sharded_contiguous"]

class DramAddressGenerator:
    def tile(self, logical_tile_index: RuntimeValue) -> DramEndpoint: ...
    def span(self, first_tile: RuntimeValue, count: RuntimeValue) -> tuple[DramTransfer, ...]: ...

```

It must derive bank, bank-local page, NoC coordinate, address, burst chunks, and partial-tail behavior from the layout descriptor. Runtime and compiler use the same versioned formula; golden host tests compare generated addresses to a pure Python reference.

8.6 CB/L1 allocation and Dst policy

CB backing allocation uses the deterministic lifetime allocator from section 6. Reserve additional named L1 arenas for barriers, scalar scratch, and compiler spills; do not place them by magic addresses in templates. The current rewrite validates only that each CB individually lies in the data region (`blackhole-py-rewrite/program.py:183-190`); the bundle verifier must also check overlap among all CB/scratch/barrier allocations.

[U] Dst capacity and FP32/BF16 accumulation limits are not established authoritatively by included current code. Until exact local evidence exists, algorithms requiring more than the smallest demonstrated Dst footprint or unverified formats remain disabled. Capability entries are populated by source-backed tests/documentation later. Numerical mode is explicit per recipe: correctness work should prefer FP32 accumulation where verified, then compare BF16/HiFi2/LoFi variants against CPU references before making performance defaults. The old model explicitly chooses HiFi2 because accumulated LoFi error was unacceptable across layers (`blackhole-py/examples/llama3/model.py:35-38`); that is historical application evidence, not a universal hardware specification.

8.7 Buffer and scalar binding

The rewrite parameter table currently holds one 32-bit address per Param; `Common.param` loads that word (`blackhole-py-rewrite/ttk/common.py:166-169`). **[P]** Keep tensor addresses in a stable table and add a distinct scalar ABI rather than overloading buffer identity:

```
Tensor address table: fixed ordered uint32 device addresses
Scalar launch table: typed uint32/uint64 bit patterns and bounded symbolic values
Per-core partition table: first tile, tile count, valid bounds, optional shard coordinates
```

`start_pos` in the KV probe is a bounded symbolic PARAM, not a buffer (`uop-dumps/generate_uop_dumps.py:246-252`). Tinygrad `ProgramInfo.vars/vals` already carry symbolic launch values (`tinygrad/tinygrad/uop/ops.py:1104-1112`; `tinygrad/tinygrad/engine/realize.py:182-185`). The TT runtime maps `vals` to the artifact's scalar slots, validates bounds, derives per-core spans/masks, and writes launch tables without recompiling.

The binding contract must reject:

- a parameter object not owned by the artifact;
- wrong dtype, logical shape, physical shape, layout, access mode, or alias class;
- an address outside the ABI width/window;
- a scalar outside declared bounds;
- a cache buffer whose persistent layout/version differs from the compiled mutation recipe.

The attached `Program.bind` patch implements the first useful subset for the current rewrite (`research-output/first_patch.diff:1-65`).

8.8 Runtime queueing and completion

[P] Initial runtime: use the rewrite's current upload/launch/CQ path, one Tinygrad program per synchronous `cq.submit`. This is simplest for correctness and preserves Tinygrad's scheduled order. Cache uploaded images by artifact hash, so later launches only write parameters/scalars/barriers plus Run.

[P] Next runtime step: port the predecessor’s ability to lower several dependent programs and submit one ordered CQ stream (`blackhole-py/device.py:100-115`, `blackhole-py/device.py:209-216`; `blackhole-py/cq.py:427-439`). Tinygrad’s `LINEAR` already gives the order. The runtime can batch adjacent TT calls until a host-visible copy, synchronization request, incompatible queue, or allocation hazard. Trace capture/replay can then key on the sequence of artifact hashes plus stable buffer/scalar slots, following the old source’s capture/replay model (`blackhole-py/device.py:116-150`).

The queue contract must preserve:

- all image/parameter writes before GO;
- program-to-program completion or explicit device-side dependency;
- barrier/counter reset before reuse;
- write acknowledgments before a consuming program reads DRAM;
- event/timestamp association with the correct Tinygrad call;
- timeout diagnostics that include artifact, core, role, and last phase.

8.9 Mixed-device copies, subviews, and JIT replay

Ingress/egress `COPY` roots should remain ordinary Tinygrad `COPY` calls unless the TT lowerer has a supported direct transfer path. `exec_copy` already chooses transfer/map/copyin alternatives based on source/destination allocators (`tinygrad/tinygrad/engine/realize.py:162-174`). TT allocator transfer support can later optimize TT ↔ host or TT ↔ TT copies without changing graph semantics.

Symbolic shapes/positions compile against bounds. JIT replay supplies current values through `vals`; runtime derives actual tile count and masks. A symbolic value can affect launch tables but not artifact structure unless it exceeds the compiled maximum or selects a different algorithm/layout. Such a violation is a clean cache miss/recompile, not unchecked execution.

Persistent KV caches should be explicit read-write tensor ports with mutation output states. Their allocation survives calls/JIT traces, and `AFTER(cache, call)` becomes the state consumed by subsequent attention programs. Aliased views of the cache are legal only when the artifact’s tile/address map proves the accessed region.

9. Worked lowering example: one-program RMSNorm

This example is a **[P] lowering design**, grounded in the current F32 probe. It demonstrates the requested path from shaped UOps to roles, CBs, parameters, phases, and final Tinygrad call graph. It is not currently emit-able end to end because rewrite TTK lacks reduction, reciprocal-square-root, multiply, exact scalar handoff, and verified Dst primitives.

9.1 Input graph and current generic split

The probe is:

```
x = Tensor.empty(1, 1, 2048, dtype=float32, device="PYTHON")
weight = Tensor.empty(2048, dtype=float32, device="PYTHON")
y = x * (x.square().mean(-1, keepdim=True) + 1e-5).rsqrt() * weight
```

(uop-dumps/generate_uop_dumps.py:226-229). After callify the inner shaped function still expresses the reduction and pointwise product. Generic Tinygrad creates two kernels: one scalar reciprocal RMS (uop-dumps/current/rmsnorm/50_kernel_00_base_ast.txt:4-22) and one 2048-element multiply (uop-dumps/current/rmsnorm/50_kernel_01_base_ast.txt:4-22).

Conceptual callified UOps:

```
inv_rms = RECIPROCAL(SQRT(REDUCE_ADD(x * x, axis=-1) * (1/2048) + 1e-5))
y_value = x * inv_rms * weight
STORE(out, y_value)
AFTER(out, STORE(...))
```

9.2 Logical and physical mapping

Logical input/output shape is (1, 1, 2048). Treating the last two dimensions as a row matrix gives logical (1, 2048). Correctness-first physical padding is (32, 2048), or 64 tiles of 32×32. Only one row per tile is valid; the other 31 rows are masked. F32 tile/page size is 4096 bytes under the rewrite's dtype/page formula (blackhole-py-rewrite/program.py:15-20, blackhole-py-rewrite/program.py:65-74).

Initial partition: one core owns tile columns [0, 64). This avoids inventing an unverified cross-core reduction. Later, a verified reduction collective may shard columns and combine partial sums.

9.3 Region and resource plan

```
Tensor ports
  0 out      write  F32 logical (1,1,2048), physical (32,2048), tiled
  1 x        read   F32 logical (1,1,2048), physical (32,2048), tiled
  2 gamma    read   F32 logical (2048),      physical (32,2048), tiled/broadcast-row

Scalar ports
  eps        compile-time F32 = 1e-5
  inv_dim     compile-time F32 = 1/2048

CBs (depth 2)
  cb_x        2 × 4096 = 8192 bytes, BRISC -> TRISC0, phases 0 and 2
```

```
cb_gamma  2 × 4096 = 8192 bytes, BRISC -> TRISC0, phase 2
cb_out    2 × 4096 = 8192 bytes, TRISC2 -> NCRISC, phase 2
```

Named scratch

```
inv_rms   one F32 scalar or one compiler-defined scalar tile slot
```

External ports: 3

CB IDs: 3

Peak declared CB backing: 24 KiB

Core span: core[0] = [0,64)

Code bytes, local bytes, Dst use, SFPU LRegs, replay entries, and exact scalar scratch mechanism are filled by template dry-run. The program is illegal until those values are known and within verified capabilities.

9.4 Phase plan and role pseudocode

Phase 0: stream x and accumulate sum of squares

BRISC:

```
for tile in 0..63:
    cb_x.reserve_back()
    noc.read(x.tile(tile), cb_x.write_ptr, 4096)
    wait read completion
    cb_x.push_back()
```

TRISC0:

```
for tile in 0..63:
    cb_x.wait_front()
    unpack x tile to SrcA with invalid rows zeroed
    cb_x.pop_front()
```

TRISC1:

```
initialize FP32 accumulator = 0
for tile in 0..63:
    square SrcA valid lanes
    reduce valid row into accumulator
inv_rms = rsqrt(accumulator * (1/2048) + eps)
store inv_rms in a declared TRISC1-local/scalar scratch representation
```

TRISC2/NCRISC:

```
no output; participate in phase transition protocol as required
```

Phase 1: barrier/state transition

```
all active roles reach reduce_complete
ensure Dst/math result and scalar representation are committed
establish unpack/math/pack state for elementwise output
```

Phase 2: reread x and gamma, scale, pack, write

BRISC:

```
for tile in 0..63:
    read x.tile(tile) -> cb_x
    read gamma.tile(tile) -> cb_gamma
```

```

TRISC0:
    unpack x -> SrcA
    unpack gamma -> SrcB

TRISC1:
    y = SrcA * broadcast(inv_rms) * SrcB
    mask invalid rows
    publish one Dst tile

TRISC2:
    acquire Dst
    cb_out.reserve_back()
    pack F32 tile to cb_out
    cb_out.push_back()

NCRISC:
    cb_out.wait_front()
    noc.write_ack(cb_out.read_ptr, out.tile(tile), 4096)
    cb_out.pop_front()

```

The scalar handoff is intentionally explicit as an unresolved template requirement. Current TTK has eight SFPU LRegs and replay machinery but no RMSNorm operations ([blackhole-py-rewrite/ttk/sfpu.py:63-153](#)). The implementation must prove whether `inv_rms` can remain in a TRISC1 architectural register/LReg across the phase, or define an L1 scratch/reload sequence. It must not assume undocumented persistence.

9.5 Bundle construction sketch

```

ports = (
    TensorPort(0, "out",    "write", F32, (1,1,2048), (32,2048), TT_TILED),
    TensorPort(1, "x",      "read",  F32, (1,1,2048), (32,2048), TT_TILED),
    TensorPort(2, "gamma",  "read",  F32, (2048,),      (32,2048), TT_TILED),
)

bundle = TTBundleBuilder(cores=(core0,), tensor_ports=ports, scalar_ports=(), caps=caps)
cb_x = bundle.cb(CBSpec("x", F32, 4096, 2, "brisc", "trisc0", phases=(0,2)))
cb_g = bundle.cb(CBSpec("gamma", F32, 4096, 2, "brisc", "trisc0", phases=(2,2)))
cb_o = bundle.cb(CBSpec("out", F32, 4096, 2, "trisc2", "ncrisc", phases=(2,2)))
reduce_done = bundle.barrier(BarrierSpec("reduce_done", parties=5))

bundle.emit("brisc", RMSNormReader(x=ports[1], gamma=ports[2], cb_x=cb_x, cb_gamma=cb_g))
bundle.emit("trisc0", RMSNormUnpack(cb_x=cb_x, cb_gamma=cb_g, valid_rows=1))
bundle.emit("trisc1", RMSNormMath(width=2048, eps=1e-5, inv_dim=1/2048, barrier=reduce_done))
bundle.emit("trisc2", RMSNormPack(cb_out=cb_o, valid_rows=1, barrier=reduce_done))
bundle.emit("ncrisc", RMSNormWriter(out=ports[0], cb_out=cb_o))

artifact = bundle.lower()          # exact code/resource verification occurs here

```

9.6 Final ordinary Tinygrad UOps

```
semantic = SINK(AFTER(out_param, STORE(out_param, x * rsqrt(mean(x*x)+eps) * gamma)))
program  = PROGRAM(semantic, empty LINEAR, manifest SOURCE, role-image BINARY,
                  ProgramInfo(globals=(0,1,2), outs=(0,), ins=(1,2), ...))
call     = CALL(program, out_param, x_param, gamma_param)
result   = AFTER(out_param, call)
return SINK(result)
```

Tinygrad then calls `create_schedule` on this graph. It sees one producer call and one output state; no special TT scheduling path is required. Runtime validates three bound buffers, ensures tiled backing, uploads the artifact once, writes addresses, launches, and marks output tiled state current.

9.7 Correctness tests and diagnostics

Before hardware execution, host tests should verify:

- recipe matcher accepts only the exact RMSNorm structure and dimension/axis contract;
- logical-to-physical mask has 64 tiles and one valid row each;
- partition is deterministically `[0, 64)`;
- CB allocation is stable and non-overlapping;
- generated role images are deterministic and fit fixed text/local ranges;
- the artifact's semantic UOp fingerprint matches the input region;
- changing buffer addresses leaves the artifact key unchanged;
- changing dtype, width, layout policy, numerical mode, or recipe source changes the key;
- malformed binding is rejected.

On device later, compare against Tinygrad/NumPy F32 reference for adversarial magnitudes, zeros, denormals as supported, and random inputs; inspect intermediate scalar through a debug variant; report max absolute/relative error and exact numerical mode. A resource failure should name the limiting role/CB/Dst/state fact rather than "RMSNorm unsupported."

10. Llama coverage and gap matrix

The two Blackhole trees answer different questions. The rewrite demonstrates a smaller, cleaner five-role assembly/runtime substrate; the predecessor contains many hand-authored Llama kernels and model orchestration. Neither tree contains a compiler that accepts the current Tinygrad callified Llama graph and emits those kernels. The table therefore distinguishes **semantic coverage**—an operation appears in executable source—from **direct-lowering coverage**—the current Tinygrad UOp form can be recognized, partitioned, compiled, bound, and launched automatically.

Llama capability	Current Tinygrad/corpus evidence		historical evidence	Direct-lowering gap and first useful acceptance test
One-tile elementwise and five-role plumbing	The shaped graph can retain arbitrary pointwise arithmetic until materialization; the one-call matmul epilogue shows pointwise operations are not intrinsically separate kernels (uop-dumps/current/matmul_epilogue/50_kernel_00_base_ast.txt:4-34).	[I] Add1 has a BRISC reader, TRISCO unpack, TRISC1 SFPU add, TRISC2 pack, NCRISC writer, CBs, and a barrier (blackhole-py-rewrite/examples/add1.py:20-71). Host lowering emitted all five role images (research-output/add1_host_lowering.out:1-8).	[H] Numerous elementwise role kernels exist, but they use predecessor-specific builders and runtime arguments.	[P] No UOp matcher, layout analysis, tile iterator, or PROGRAM bridge exists. First test: lower $(x + 1).realize()$ on a single padded tile to one complete PROGRAM while comparing the semantic graph and manifest on CPU; no hardware result is required for this milestone.
Multi-tile elementwise, broadcast, partial tiles	Tinygrad preserves movement and broadcast indexing in the shaped/callified graph and only later produces range/reduction kernels (tinygrad/tinygrad/schedule/rangeify.py:574-619).	[I, narrow] KernelBundle can specialize each selected core/role and validates CB/L1 bounds, but the add1 example is one fixed BF16 tile (blackhole-py-rewrite/program.py:171-207; blackhole-py-rewrite/examples/add1.py:15-18).	[H] Residual add deterministically shards tiles over live cores and builds all five roles (blackhole-py/examples/llama3/residual.py:203-251).	[P] Define logical-to-tile indexing, masks, scalar/row/column broadcasts, stable core spans, and a layout contract. First test: non-multiple-of-32 shapes with row broadcast; verify every logical element is covered once, padded lanes are masked, and resource manifests are deterministic.
RMSNorm reduction and broadcast multiply	Generic Tinygrad realizes RMSNorm as two calls: a scalar reduction/sqrt/reciprocal call followed by a pointwise scale-and-weight call (research-output/corpus_inventory.out:7-15; uop-dumps/current/rmsnorm/50_kernel_00_base_ast.txt:4-22; uop-dumps/current/rmsnorm/50_kernel_01_base_ast.txt:4-22). The model expression is $x * \text{rsqrt}(\text{mean}(x*x) + \text{eps}) * \text{weight}$ (tinygrad/tinygrad/nn/__init__.py:296-304).	[missing] Current Math has no reduction or multiply template, and current SFPU exposes only a small operation subset (blackhole-py-rewrite/ttk/math.py:6-44; blackhole-py-rewrite/ttk/sfpu.py:63-153).	[H] A one-core five-role RMSNorm program explicitly emits square reduction, reciprocal square root, row-broadcast weight multiply, CB staging, and launch arguments (blackhole-py/examples/llama3/rmsnorm.py:416-713, blackhole-py/examples/llama3/rmsnorm.py:1045-1095). Its buffer wrapper validates physical shapes before launch (blackhole-py/examples/llama3/rmsnorm.py:1098-1130).	[P] Port only the proven instruction templates after defining Dst/LReg lifetime rules; recognize the UOp formula rather than calling the historical constructor. First test: the worked example in section 9, including adversarial values, exact masks, deterministic resource report, and a refusal when scalar persistence or L1 cannot be proved.

Llama capability	Current Tinygrad/corpus evidence		historical evidence	Direct-lowering gap and first useful acceptance test
Matmul plus pointwise epilogue	matmul_epilogue is already one generic call containing reduction, bias add, maximum/ReLU, and store (research-output/corpus_inventory.out:1-5; uop-dumps/current/matmul_epilogue/50_kernel_00_base_ast.txt:4-34). Llama projection calls likewise contain neighboring normalization, RoPE, or residual arithmetic rather than splitting every pointwise op (research-output/corpus_inventory.out:41-97).	[missing] No matmul planner or unpack/math/pack matmul templates exist in the rewrite (blackhole-py-rewrite/ttk/unpack.py:12-50; blackhole-py-rewrite/ttk/math.py:6-44; blackhole-py-rewrite/ttk/pack.py:5-48).	[H] The Llama path invokes the predecessor's matmul planner for Q/K/V and MLP projections (blackhole-py/examples/llama3/attn.py:364-414; blackhole-py/examples/llama3/mlp.py:113-181).	[P] Port a versioned matmul recipe and numeric-mode description, not the whole old runtime. First test: matmul_epilogue must become one bundle when resource-legal, with bias and ReLU executed before the final materialization; compare F32/BF16 modes separately.
RoPE	The full block's projection/RoPE region is visible before generic rangeification, and current schedule call 3/4 already combine RoPE-related pointwise work with V/K-cache or QK work (research-output/corpus_inventory.out:49-61).	[missing] Current SFPU lacks the multiply/add/permute repertoire and no tiled RoPE recipe exists.	[H] The predecessor builds a five-role RoPE program and, in cache mode, names it llama3_rope_kv_store; its decode wrapper validates position, compact tables, projection dimensions, and complete cache-buffer tuples (blackhole-py/examples/llama3/attn.py:1060-1075, blackhole-py/examples/llama3/attn.py:1078-1110).	[P] Infer head/half-head indexing and table access from UOps, then choose whether K RoPE and KV update share a bundle. First test: fixed and symbolic positions, Q and K paths, edge positions at tile boundaries, and a manifest that declares every mutated cache page.
Symbolic in-place KV-cache update	kv_cache_update_symbolic remains one call with a symbolic start_pos; the cache effect is represented as STORE feeding AFTER, not as a separate opaque side channel (research-output/corpus_inventory.out:35-39; uop-dumps/current/kv_cache_update_symbolic/50_kernel_00_base_ast.txt:4-47). The full shaped block carries the same cache STORE → AFTER edge (uop-dumps/current/llama32_1b_block_decode/00_shaped_sink.jsonl:159-162).	[missing] Buffer has no strides/layout/alias descriptor, Program.bind originally validates only an address, and there is no symbolic tile-address generator (blackhole-py-rewrite/program.py:22-41, blackhole-py-rewrite/program.py:94-101; blackhole-py-rewrite/ttk/dram.py:1-21).	[H] RoPE/cache code has an actual cache-mode path, but its address formulas and assumptions are hand-authored for the model.	[P] Preserve the exact AFTER mutation token in the returned graph; runtime must bind a cache port with layout and writable-region metadata. First test: two launches at different start_pos values reuse one artifact, write disjoint expected tiles, and produce a schedule dependency for a following cache read.
GQA score matmul	sdpa_gqa_decode and the block expose QK reduction with symbolic sequence length before the max/sum/probability calls (research-output/corpus_inventory.out:17-33, research-output/corpus_inventory.out:54-65).	[missing] No head-group partitioner, score matmul recipe, dynamic page loop, or score-layout contract.	[H] The score builder requires one core per KV head, validates group size and logical/padded time, constructs a one-score-tile matmul plan, and emits all five roles and CBs (blackhole-py/examples/llama3/attn.py:1570-1640).	[P] Recognize GQA head sharing and symbolic live pages from indexing rather than model names. First test: multiple legal start_pos values and GQA ratios must produce stable code with launch-time page bounds; reject a ratio or head tile that exceeds the declared capability.

Llama capability	Current Tinygrad/corpus evidence		historical evidence	Direct-lowering gap and first useful acceptance test
Stable softmax	Decode SDPA currently becomes four calls: score/initial reduction, row max, reciprocal of exp-sum, probability materialization, then probability×V (research-output/corpus_inventory.out:17-33; uop-dumps/current/sdpa_gqa_decode/50_kernel_00_base_ast.txt:4-43; uop-dumps/current/sdpa_gqa_decode/50_kernel_01_base_ast.txt:4-21; uop-dumps/current/sdpa_gqa_decode/50_kernel_02_base_ast.txt:4-31; uop-dumps/current/sdpa_gqa_decode/50_kernel_03_base_ast.txt:4-54).	[missing] No max/sum reduction, exp2, reciprocal, mask, or multi-pass streaming template.	[H] The predecessor's streaming FP32 softmax states that eight cores own four rows each and retain max/sum in SFPU registers (blackhole-py/examples/llama3/softmax.py:1-8). Its builder validates page count/core count and builds three CBs and five roles (blackhole-py/examples/llama3/softmax.py:483-525).	[P] Decide fusion from explicit state/resource accounting. A first backend may use two or three programs rather than promise one: score+max, exp/sum+normalize, PV. Tests must compare stable-reference output at extreme logits and verify masked tail columns are exactly excluded.
Probability×V and output projection/residual	Generic call 8 performs probability×V reduction; call 9 combines output projection and residual add (research-output/corpus_inventory.out:74-81).	[missing] No matmul or GQA value-gather recipe.	[H] Attention and residual kernels exist as separately launched programs; the block calls them in sequence (blackhole-py/examples/llama3/block.py:89-120).	[P] Implement a value-gather/PV recipe, then allow output projection epilogue to write the residual result directly when layouts, aliasing, and resource limits permit. Test both the fused and forced-cut paths against one semantic graph.
SwiGLU and MLP	The raw block has one explicit FFN gate CONTIGUOUS; removing it reduces 14 calls to 13, proving one current boundary is model-imposed rather than required by generic scheduling (tinygrad/tinygrad/llm/model.py:100-121; research-output/fusion_boundary_experiment.out:1-12). Calls 11-13 are gate+SiLU, up×gate, and down+residual (research-output/corpus_inventory.out:86-97).	[missing] SFPU lacks SiLU/exp/reciprocal/multiply, and no paired projection/streaming MLP planner exists.	[H] A fused SwiGLU program deterministically shards tiles and emits five roles (blackhole-py/examples/llama3/swiglu.py:283-357); the historical MLP still launches gate matmul, up matmul, fused SwiGLU, and down matmul separately (blackhole-py/examples/llama3/mlp.py:113-181).	[P] First remove/ignore only the explicit temporary CONTIGUOUS when legality is proven; do not assume gate and up projections fit one bundle. Test one-program SwiGLU, down+residual epilogue, and deterministic cuts when projection or CB resources overflow.
One decode block	Current Tinygrad creates one opaque precompiled block call whose internal schedule contains 14 generic calls (tinygrad/tinygrad/llm/model.py:130-137; research-output/corpus_inventory.out:41-97).	[missing] No Tinygrad integration or block planner.	[H] The predecessor orchestrates RMSNorm → attention → residual → RMSNorm → MLP → residual across existing buffers (blackhole-py/examples/llama3/block.py:39-149). This is a sequence of hand-built programs, not a fused compiler result.	[P] Acceptance is semantic and structural: lower the unmodified corpus graph, preserve cache effects and symbolic position, emit a deterministic program DAG, and compare host-reference outputs. Program count is diagnostic, not pass/fail; five-to-eight is a hypothesis only.

Llama capability	Current Tinygrad/corpus evidence		historical evidence	Direct-lowering gap and first useful acceptance test
Full prefill/decode, resident weights/KV, batching, trace replay	Tinygrad corpus covers one block only; it does not establish full-model execution.	[missing] Rewrite <code>Device.run</code> submits each program synchronously, and there is no graph capture/replay (<code>blackhole-py-rewrite/device.py:112-123</code> ; <code>blackhole-py-rewrite/cq.py:223-244</code>).	[H] <code>SequentialModel</code> allocates resident per-layer state and requires at least eight cores (<code>blackhole-py/examples/llama3/model.py:116-160</code>). It implements sequential prefill and batched decode (<code>blackhole-py/examples/llama3/model.py:581-660</code>), final norm/LM head/argmax (<code>blackhole-py/examples/llama3/model.py:662-700</code>), and trace capture/replay (<code>blackhole-py/examples/llama3/model.py:731-768</code>).	[P] First compile one block and execute a normal ordered program DAG. Only then port batched CQ submission, upload deduplication, and capture/replay. Full-model claims require device evidence that is absent from this investigation.

10.1 What may be ported from `blackhole-py`

The historical tree is most valuable as a **template/evidence quarry**:

- instruction sequences and synchronization protocols for matmul, reduction, RoPE, softmax, residual, SwiGLU, and cache writes;
- deterministic sharding examples, such as residual's quotient/remainder split (`blackhole-py/examples/llama3/residual.py:213-222`) and SwiGLU's interval formula (`blackhole-py/examples/llama3/swiglu.py:283-292`);
- runtime concepts absent from the rewrite, especially queued multi-program fast dispatch and capture/replay (`blackhole-py/device.py:82-150`, `blackhole-py/device.py:209-216`);
- concrete failure checks for shapes, core counts, GQA ratios, sequence bounds, and buffer dtypes.

It should **not** be ported as a monolithic compiler layer. Its kernels bake in model dimensions, CB IDs, scratch addresses, core counts, runtime-argument order, and numeric modes. Those facts must become explicit versioned capabilities, template parameters, or recipe constraints. The direct backend should derive semantic operands and effects from the Tinygrad graph; historical names such as `llama3_rmsnorm` or `llama3_attention_scores_stream` must not be its matching interface.

10.2 Evidence ceiling

The matrix establishes that substantial hand-written functionality exists, but it does not establish silicon correctness, performance, or portability. The bundle omits TT-Metal, TT-LLK, the referenced historical branch/stash, and other external sources (`metadata/EXCLUSIONS.md:23-31`). Therefore this report does not certify comments that attribute register formats, reset behavior, port counts, or instruction semantics to those omitted sources. Any imported historical sequence remains **[H]** until it passes host structural checks and later device comparison tests under the rewrite runtime.

11. Phased implementation and test plan

The order below deliberately advances one semantic/resource dimension at a time. Every milestone must leave behind (1) a CPU/reference comparison where meaningful, (2) a deterministic host-only artifact test, (3) a negative test that exercises a named diagnostic, and (4) a device test specification even when no device is available. No phase should claim support merely because assembly was emitted.

Milestone 0 — Freeze the seam, ABI, and evidence artifacts

Files and components. Land the generic graph-lowering hook represented by `research-output/architecture_hook.diff`; harden `blackhole-py-rewrite/program.py::Program.bind` with `research-output/first_patch.diff`; add an immutable capability record and bundle-manifest schema under the rewrite, for example `blackhole-py-rewrite/target.py` and `blackhole-py-rewrite/artifact.py`. Retain the UOp generator and selected probes as regression fixtures (`uop-dumps/generate_uop_dumps.py:147-188`, `uop-dumps/generate_uop_dumps.py:215-323`).

Host/reference tests. The hook's four focused tests establish unchanged default behavior, complete root delivery, backend cache separation, and deterministic multiple-claim failure (`research-output/architecture_hook.diff:83-152`; `research-output/architecture_hook_test.out:1-9`). Six selected assignment/precompiled/custom-kernel regressions pass (`research-output/architecture_hook_selected_regression.out:1-14`). The binding patch's four tests cover valid replacement, foreign parameter identity, ABI mismatch, and address range (`research-output/first_patch.diff:22-65`; `research-output/first_patch_test.out:1-9`). Renderer-neutral regeneration must remain byte-identical for the five central probes (`research-output/uop_renderer_neutral_regeneration.out:1-6`).

Failure diagnostics. Reject an empty cache tag, multiple lowerer claimants, a graph with unsupported root devices, foreign runtime parameters, or a buffer whose dtype/logical shape/padded shape/layout/address width differs from the artifact port. Report the root index and offending contract field.

Device test specification. None yet. The milestone is complete without opening PCIe.

Milestone 1 — TT device plumbing and a complete prebuilt PROGRAM

Files and components. Add `tinygrad/tinygrad/runtime/ops_tt.py` with `TTDevice`, allocator, compiler/runtime callable, and target metadata; add an inert TT renderer class only to satisfy current `pm_compile`, which still asks `Device[call.device].renderer` before `to_program` can preserve a prebuilt `PROGRAM` (`tinygrad/tinygrad/engine/realize.py:244-247`, `tinygrad/tinygrad/codegen/__init__.py:223-257`). Add a rewrite-side adapter that loads a `TTBundleArtifact`, binds Tinygrad buffers/scalars, emits upload/bind/launch commands, and returns completion. Keep scheduling pure: `Device[...]` dynamically imports and constructs a device and must not be touched by the graph lowerer (`tinygrad/tinygrad/device.py:14-35`).

Host/reference tests. Construct a synthetic `PROGRAM` with five return-only role images and verify `CALL(PROGRAM)` reaches the TT runtime mock with ordered globals, outputs, inputs, and symbolic values matching `ProgramInfo` (`tinygrad/tinygrad/uop/ops.py:1090-1135`; `tinygrad/tinygrad/engine/realize.py:111-119`, `tinygrad/tinygrad/engine/realize.py:176-186`). Verify compile-cache identity excludes concrete addresses and launch values but includes artifact/capability versions.

Failure diagnostics. "Artifact target mismatch," "missing role image," "port 2 expected TT tiled BF16 (32, 64)," "symbol start_pos outside compiled range," and "firmware/ABI version mismatch" must be distinct errors.

Device test specification. Boot firmware and launch only return-only workers; verify completion event and no data mutation. This remains future work here.

Milestone 2 — Generate one-tile add1 from shaped UOps

Files and components. Add `blackhole-py-rewrite/tinygrad_lowering.py` with a first pure matcher/recipe; add `analysis.py` records for logical/physical shape and access maps; wrap the existing add1 TTK sequence as a parameterized template rather than calling the example script. Return one PROGRAM and one AFTER output state.

Host/reference tests. Match the exact pointwise graph, lower it twice, and require byte-identical artifact/manifest output. Compare the artifact's semantic-source fingerprint to the callified region. Verify the generated role images and parameter/CB map agree structurally with the independently lowered example (`blackhole-py-rewrite/examples/add1.py:20-71`; `research-output/add1_host_lowering.out:1-8`).

Failure diagnostics. Reject an unsupported dtype, more than one tile, non-contiguous physical policy, or unrecognized arithmetic with a diagnostic that identifies the first unsupported UOp and candidate recipe.

Device test specification. BF16 one-tile add1 against NumPy/Tinygrad, including infinities, NaNs according to selected numeric policy, and repeated launches with different buffers but one uploaded artifact.

Milestone 3 — Multi-tile elementwise, broadcasts, masks, and deterministic partitioning

Files and components. Add a tile-access normalizer, mask generator, `CoreSpan` planner, and generic streaming elementwise templates. Extend `TTBundleArtifact` with logical/padded shapes, layouts, ordered tile spans, and per-core launch arguments. Use stable sorted core coordinates and quotient/remainder intervals as specified in section 6, not Python object identity or dynamic timing.

Host/reference tests. Cover contiguous and movement-derived row/column/scalar broadcast, shapes smaller than a tile, and non-multiple-of-32 tails. Enumerate every logical element to prove exact coverage and every padded lane to prove masking. Reorder input dictionary construction and require identical artifact hashes. Use historical residual sharding only as a cross-check (`blackhole-py/examples/llama3/residual.py:213-248`).

Failure diagnostics. Explain “unsupported non-affine tile map,” “partial-tile store lacks mask,” “core span empty/overlapping,” or the exact L1/CB/code limit exceeded.

Device test specification. Random and adversarial shapes across one and several cores, compared after inverse tilization.

Milestone 4 — Reduction and one-program RMSNorm

Files and components. Port the minimum reduction, square, add, reciprocal-square-root, multiply, row-broadcast, and phase/barrier templates into current TTK with explicit ownership. Implement the section 9 recipe and verifier. Do not silently rely on predecessor scratch addresses or persistent registers.

Host/reference tests. Recognize both standalone RMSNorm and the normalization prefix embedded in Llama projection roots. Produce one program only when scalar/LReg/L1/CB/phase constraints prove legal; otherwise emit the current two-call materialization or a named refusal. Compare formula-level results with F32 reference and test deterministic manifest/code generation. The raw generic two-call shape is the oracle for operands and effect boundaries (`research-output/corpus_inventory.out:7-15`).

Failure diagnostics. Identify whether failure is unsupported reduction axis, width, Dst capacity, LReg persistence, CB lifetime, code size, numeric mode, or missing partial-row mask.

Device test specification. RMSNorm at widths 32 through 2048, zero and large/small magnitude vectors, error bounds recorded separately for reduction and final BF16 pack.

Milestone 5 — Matmul plus epilogue

Files and components. Port a single, versioned matmul block planner and unpack/math/pack templates from the historical implementation into rewrite TTK. Add recipe matching for multiply-reduce plus bias/ReLU/residual epilogues. The recipe must declare input/output/intermediate formats and Dst accumulation mode.

Host/reference tests. Lower `matmul_epilogue` to one bundle and verify its final semantic STORE still includes bias and maximum (`uop-dumps/current/matmul_epilogue/50_kernel_00_base_ast.txt:4-34`). Sweep legal tile dimensions, compare planner coverage, and force each resource cut. Verify the same artifact is reused across address changes.

Failure diagnostics. Name incompatible K blocking, unsupported stride/layout, Dst accumulator overflow, CB page budget, role code overflow, or epilogue op unsupported.

Device test specification. Small matrices with high-precision reference, then Llama projection dimensions; report numeric mode and per-stage error rather than only an end-to-end token match.

Milestone 6 — RoPE and symbolic KV mutation

Files and components. Add head-aware affine indexing, compact sin/cos table ports, writable cache regions, symbolic launch parameters, and mutation effects. Implement Q RoPE and K RoPE+V/K cache write recipes. Returned Tinygrad graph must attach every mutated cache buffer to the same CALL through AFTER.

Host/reference tests. Use `kv_cache_update_symbolic` as the minimal mutation probe and the full block's cache edge as integration coverage (`uop-dumps/current/kv_cache_update_symbolic/50_kernel_00_base_ast.txt:4-47`; `uop-dumps/current/llama32_1b_block_decode/11_calified_inner_sink.jsonl:162-163`). Compile once and bind several `start_pos` values; verify artifact key stability, address formulas, writable regions, and schedule ordering with a subsequent read.

Failure diagnostics. “Symbolic bound changes compiled tile count,” “cache write escapes declared region,” “aliasing output has incompatible layout,” and “mutation root not represented in returned graph” must be explicit.

Device test specification. Step positions across 31/32 and 63/64 boundaries, read back only touched tiles, and compare Q/K RoPE plus K/V cache contents.

Milestone 7 — GQA score, stable softmax, and probability×V

Files and components. Add GQA head-group analysis, dynamic live-page loops, score matmul, max/sum reductions, exp/reciprocal, tail masking, and PV value gather. Let the deterministic planner choose one or several programs based on verified SFPU/LReg/Dst/CB/code and inter-phase persistence.

Host/reference tests. Lower `sdpa_gqa_decode` and require semantic equivalence to the four generic calls, not a preset program count (`research-output/corpus_inventory.out:17-33`). Exercise sequence lengths around page boundaries and extreme logits; compare to stable F32 softmax and ensure masked columns contribute neither max nor sum. Verify deterministic cuts and explain every cut edge.

Failure diagnostics. Separate illegal GQA ratio, page-count range, unsupported mask, persistent-state shortage, reduction-tree shortage, PV layout mismatch, and external-port limit.

Device test specification. Compare score, max, denominator, probability, and PV debug variants before enabling more aggressive fusion.

Milestone 8 — SwiGLU/MLP and explicit materialization removal

Files and components. Add SiLU and paired-elementwise templates, recognize gate/up/down projection structure, and treat CONTIGUOUS as removable only if its buffer identity is not externally observed and the fused consumer accepts the producer layout. Preserve an option to force materialization for diagnostics.

Host/reference tests. Reproduce the 14 → 13 schedule change by eliminating the FFN gate temporary while keeping all semantics (`research-output/fusion_boundary_experiment.out:1-12`). Lower standalone SwiGLU to one bundle, then test gate+SiLU, up×gate, and down+residual combinations under resource pressure. Compare to the historical four-stage MLP only as behavioral evidence (`blackhole-py/examples/llama3/mlp.py:113-181`).

Failure diagnostics. State whether a cut is due to explicit observable materialization, incompatible layout, two-projection input bandwidth, CB/L1/Dst/code limit, or missing SiLU numeric mode.

Device test specification. Standalone SwiGLU and full MLP against F32/BF16 references across saturation ranges.

Milestone 9 — One unmodified Tinygrad Llama decode block

Files and components. Integrate all recipes in a region enumerator, legal cut solver, and ordinary call-graph emitter. Add a block artifact report containing root ownership, chosen regions, rejected fusion edges, resources, layouts, mutations, and cache keys. Do not special-case the probe name.

Host/reference tests. Feed the exact regenerated `llama32_1b_block_decode` callified graph. Require all 14 generic-call semantics and cache effects to be covered exactly once, no unsupported root silently dropped, stable output across process/hash seed, and a human-readable region/cut report. Five-to-eight programs is a performance-oriented target, not a correctness assertion.

Failure diagnostics. Print a shortest unsupported UOp path and the candidate regions/cuts considered. For every materialized intermediate, report whether it is required by effect/ABI/layout/resource constraints or retained by a conservative first implementation.

Device test specification. Compare every materialized boundary against Tinygrad/NumPy for several positions, then compare final block output and cache updates. Record command count, bytes, and timings without claiming model-level performance.

Milestone 10 — Full decode/prefill and runtime optimization

Files and components. Add resident-weight/KV lifecycle, batched CQ submission, image-upload deduplication, asynchronous event bookkeeping, and eventually capture/replay. Port concepts—not object APIs—from the historical queue/capture path (`blackhole-py/device.py:82-150`, `blackhole-py/device.py:209-216`; `blackhole-py/cq.py:427-439`).

Host/reference tests. Build full-model program DAGs without device access, verify buffer-state transitions across layers/tokens, cache-key reuse across positions, and bounded runtime metadata. Compare logical outputs with a CPU Tinygrad model where tractable.

Failure diagnostics. Distinguish compile, upload, launch, dependency, timeout, and numerical failures. A trace timeout should include current event/CQ/core/role state, following the predecessor's diagnostic intent (`blackhole-py/examples/llama3/model.py:759-767`).

Device test specification. First one layer, then several layers, then full decode; only after correctness, add sequential prefill, batching, and trace replay. Full-model throughput or latency claims require the omitted hardware evidence and are outside this report.

12. Risks, rejected alternatives, and unresolved questions

12.1 Principal technical risks

Hardware-state modeling risk. Current TTK shadow state is useful but not a complete architectural model. `Tensix` initializes many unknown configuration fields to zero, tracks context/thread/MOP/SFPU state, and emits diff writes, but it does not merge symbolic branch states (`blackhole-py-rewrite/ttk/tensix.py:182-215`, `blackhole-py-rewrite/ttk/tensix.py:247-275`). A compiler that trusts unknown-as-zero can omit required initialization. The safe policy is “unknown requires write” unless a device-reset test proves a value.

Ownership and phase risk. Five role programs execute concurrently, and CBs, Dst, unpack/math/pack engines, replay slots, SFPU LRegs, NoC streams, semaphores, and scratch have different owners and lifetimes. Current code has pieces of this discipline but no whole-bundle checker. A locally valid instruction sequence can deadlock or race when phases overlap. Every template must declare acquire/use/release facts; the verifier must build a wait-for/phase graph and reject cycles before assembly.

Numerical-mode risk. Historical kernels select BF16/FP32 input, Dst accumulation, intermediate, packer-L1, and SFPU approximations in operation-specific ways (`blackhole-py/examples/llama3/mlp.py:129-135`; `blackhole-py/examples/llama3/softmax.py:44-49`). Those choices are not inferable from dtype alone. Numerical mode must be part of recipe acceptance, artifact identity, diagnostics, and tolerance tests.

Layout/alias risk. Tinygrad Buffer allocation is fundamentally flat size/dtype/options; views depend on allocator offset support, and memory planning only considers allocators with `_offset` (`tinygrad/tinygrad/device.py:77-85`, `tinygrad/tinygrad/device.py:99-160`, `tinygrad/tinygrad/schedule/memory.py:13-16`). The rewrite’s Buffer records logical and padded shape but no strides, layout, alias set, or dirty host/device representation (`blackhole-py-rewrite/program.py:22-41`). Direct lowering therefore needs a separate explicit layout/state layer and conservative copy policy before it can safely reuse or alias buffers.

Mutation and cache risk. The graph-lowering schedule cache currently starts from normalized `function.key`, while symbolic values are removed from PARAM identity during callification (`tinygrad/tinygrad/callify.py:183-202`; `tinygrad/tinygrad/schedule/__init__.py:111-124`). That is desirable only if generated code is valid for the compiled symbolic range. Mutated buffer state must remain represented by AFTER, and artifact keys must include symbolic maxima/layout/resource policy but exclude launch-time values and addresses.

Resource-model risk. Exact role text partitions and L1 data bounds are present in the rewrite (`blackhole-py-rewrite/fw/consts.py:3-16`, `blackhole-py-rewrite/fw/consts.py:71-83`), but current sources do not fully establish usable Dst capacity, all CB/port limitations, safe replay/LReg persistence, or NoC concurrency limits. The deterministic planner must distinguish hard source-proven limits from conservative provisional limits and record the capability-table source/version.

Runtime-ordering risk. Rewrite `Device.run` submits programs one at a time and CQ waits synchronously for the final run event (`blackhole-py-rewrite/device.py:112-123`; `blackhole-py-rewrite/cq.py:223-244`). This is adequate for correctness bring-up but can hide missing explicit inter-program dependencies and will not deliver predecessor-style batching. First preserve ordinary Tinygrad call order; optimize submission only after buffer effects and completion semantics are explicit.

Source-drift/reproducibility risk. Both Blackhole trees are live working snapshots and documents can lag code (`metadata/SOURCE_STATE.md:3-16`, `metadata/SOURCE_STATE.md:106-110`). Capability tables, recipe fingerprints, and generated artifacts therefore need source hashes. Renderer-neutral corpus regeneration was stable when probes were generated in the canonical full order (`research-output/uop-renderer_neutral_regeneration.out:1-6`); process-global incidental IDs should never become semantic cache identity.

12.2 Rejected architectural alternatives

Reject historical pre-callify interception as the primary seam. It would have to reproduce callification's current semantics for explicit `CONTIGUOUS`, `COPY`, `STORE/AFTER`, precompiled multi-output functions, parameter allocation/order, symbolic cache normalization, and live-buffer state updates (`tinygrad/tinygrad/callify.py:32-52`, `tinygrad/tinygrad/callify.py:101-142`, `tinygrad/tinygrad/callify.py:169-223`). The historical TTIR proposal explicitly planned an intercept before `transform_to_call` and deferred implementation (`blackhole-py/TTIR.md:1-35`, `blackhole-py/TTIR.md:455-519`). Current source makes that duplication unnecessary.

Reject a persistent TTIR for the first backend. The historical design's tile-SSA/effect/resource records are a useful vocabulary (`blackhole-py/TTIR.md:306-376`), but no TTIR implementation exists. A second graph IR would require independent verification, serialization, cache identity, mutation semantics, diagnostics, and lowering. Short-lived immutable analysis records over current UOps provide the needed tile/effect/resource facts while returning ordinary Tinygrad calls.

Reject lowering in the scalar renderer. By renderer time the compiler is expected to render `SINK/LINEAR` into code; a prebuilt `PROGRAM` already bypasses shaped codegen, although `pm_compile` still asks for a renderer object (`tinygrad/tinygrad/codegen/__init__.py:223-257`; `tinygrad/tinygrad/engine/realize.py:244-247`). A fake scalar renderer would either be inert plumbing or would reconstruct information lost after generic rangeification. Keep it inert and put graph lowering at the scheduling seam.

Reject a Tinygrad Tensor subclass or TT-specific semantic Op. Tensor methods already produce shaped UOps (`tinygrad/tinygrad/tensor.py:108-117`), and ordinary `PROGRAM/CALL/AFTER` express opaque execution and effects (`tinygrad/tinygrad/uop/ops.py:1062-1071`). A subclass would fork front-end semantics; a new Op would fork scheduler/runtime semantics without adding necessary expressiveness.

Reject forcing one program per block or model. "One bundle," "one Tinygrad `PROGRAM`," and "one fused graph region" are different contracts (`README.md:121-137`). Effects, external outputs, layouts, core partitions, L1/CB/Dst/code/SFPU/NoC resources, and phase persistence can require cuts. The compiler's obligation is deterministic legality and explicit diagnostics, not a predetermined program count.

Reject adopting generic GPU fusion heuristics. TT's five cooperative roles, CB producer/consumer protocols, Dst/replay/LReg state, fixed role text regions, and NoC/core topology are first-class resources. Counting scalar ops or register pressure alone cannot decide legality.

Reject importing blackhole-py wholesale. Its Llama path is strong historical evidence, but it is a hand-composed sequence with fixed model shapes, CB IDs, scratch regions, role classes, RTAs, and runtime APIs (`blackhole-py/examples/llama3/block.py:39-149`). Port instruction templates, resource facts, and tested protocols behind new declarative interfaces; do not make model-specific builders the compiler ABI.

Reject treating documentation or external hyperlinks as implementation. The rewrite's `TTK.md` claims stronger binding validation and a `KernelBuilder.standalone` path than current code provides (`blackhole-py-rewrite/TTK.md:55-70`; `blackhole-py-rewrite/program.py:94-101`; `blackhole-py-rewrite/asm.py:206-239`). The `TODO` says CB and SFPU APIs are absent even though limited implementations now exist (`blackhole-py-rewrite/todo.md:36-55`, `blackhole-py-rewrite/todo.md:201-222`; `blackhole-py-rewrite/ttk/cb.py:45-112`; `blackhole-py-rewrite/ttk/sfpu.py:63-153`). Executable current source takes precedence.

12.3 Missing evidence that must remain explicit

The following questions cannot be answered authoritatively from this bundle and must not be guessed:

- What reset values, persistence rules, and context-switch behavior are guaranteed for Tensix configuration, Dst, SFPU LRegs, replay buffers, and unpack/math/pack engines on the target firmware/silicon?

- What are the exact usable Dst dimensions and aliasing rules across BF16/FP32 views for every intended numeric mode?
- What are the hard architectural CB count, external input/output port, semaphore, outstanding NoC transaction, multicast, and synchronization limits, as distinct from limits chosen by the current code?
- Which SFPU instruction approximations and edge-case semantics are guaranteed for `exp`, reciprocal, reciprocal square root, comparison/max, SiLU, and conversions, and what tolerances are expected?
- Which TT-Metal/TT-LLK initialization, address-generation, format, and barrier conventions are required but absent here?
- Which historical Llama kernels were actually run successfully on which silicon/firmware revision, with what tests and numerical results? The captured code may be executable, but this investigation did not execute it on device.
- Are comments that cite omitted Blackhole ISA or TT-Lang-generated sources exact for this captured target? Those sources are not bundled.
- What is the correct persistent layout policy for weights, activations, caches, score/probability intermediates, and host-visible outputs under Tinygrad allocation/memory planning?
- What maximum symbolic sequence range should be compiled into each attention artifact, and when must a changed bound trigger recompilation rather than only new launch values?
- Can a one-program RMSNorm, a fused score/softmax/PV region, or a fused QKV/RoPE/cache region fit and outperform cut variants on actual hardware? The current range is an unmeasured design hypothesis.

The archive explicitly omits TT-Metal, TT-LLK, the referenced historical branch and stash, and other external sources (`metadata/EXCLUSIONS.md:23-31`). The correct next evidence-gathering action is targeted host/device experimentation against a versioned capability entry, not extrapolation from a document claim.

13. Smallest useful first implementation patch

13.1 Architecture patch: a pure post-callify graph-lowering hook

The smallest architecture-enabling patch is the tested `research-output/architecture_hook.diff`. It changes only three conceptual surfaces:

1. `tinygrad/tinygrad/schedule/backend.py` — **new registry and data contract**. Add immutable `GraphLoweringRequest(function, roots)` and `BackendGraph(graph, cache_tag)`, registration/unregistration, deterministic sorted dispatch, non-empty byte cache tags, and an error when multiple lowerers claim one function (`research-output/architecture_hook.diff:35-81`).
2. `tinygrad/tinygrad/schedule/__init__.py::lower_sink_to_linear` — **one optional branch**. Call `try_lower_graph(function)` immediately before generic `get_kernel_graph`. If no lowerer claims the function, retain the exact current `function.key` and `get_kernel_graph(function)` path. If claimed, hash the normalized function key, lowerer name, and backend cache tag, then pass the returned ordinary graph to existing `create_schedule` (`research-output/architecture_hook.diff:1-34`).
3. `tinygrad/test/unit/test_backend_graph_lowerer.py` — **focused invariants**. Test unchanged default behavior, delivery of the whole callified function and every materialization root, cache separation, and deterministic multiple-claim failure (`research-output/architecture_hook.diff:82-152`).

This patch is useful before any TT recipe exists because it establishes the only Tinygrad extension point needed by the architecture while preserving all current behavior when unused. It is small enough to review independently, renderer-neutral, and does not import or instantiate a device during scheduling. Its dedicated tests pass (`research-output/architecture_hook_test.out:1-9`), as do six selected regressions around assignment, `AFTER/STORE`, nested/precompiled calls, implicit outputs, and multi-output custom kernels (`research-output/architecture_hook_selected_regression.out:1-14`). A broader full-suite claim is intentionally not made; only the recorded focused/selected runs are evidence.

The production review should consider two refinements without enlarging the first patch's scope:

- replace application-global registration with normal backend/module discovery metadata while preserving pure dispatch; and
- make roots typed `MaterializationRoot` records later, after the first TT matcher proves what additional fields are needed.

Neither refinement should move the seam or permit scheduling to open a device.

13.2 Prerequisite runtime patch: make `Program.bind` an ABI check

Before a Tinygrad runtime binds arbitrary buffers to a cached artifact, apply `research-output/first_patch.diff` to `blackhole-py-rewrite/program.py::Program.bind`. The current method only chooses the initial/replacement buffer and writes its 32-bit address (`blackhole-py-rewrite/program.py:94-101`). The patch:

- verifies the exact `Param` object belongs to the `Program`;
- compares location, dtype, logical shape, and padded shape with the compiled initial port;
- rejects addresses outside the 32-bit parameter table;
- keeps the existing unicast-write command for valid bindings (`research-output/first_patch.diff:1-20`).

The four accompanying tests pass (research-output/first_patch.diff:22-65; research-output/first_patch_test.out:1-9). This is deliberately called a prerequisite, not the architecture patch: it closes an immediate safety hole but does not enable Tinygrad graph lowering. When the layout contract is added, layout, strides/affine map, access mode, alias/mutation region, and target ABI version must join this validation.

13.3 First TT-specific follow-on patch

After the generic hook and bind hardening, the next smallest TT-specific patch should implement **only one-tile add1** end to end:

```
tinygrad/tinygrad/runtime/ops_tt.py
    TTDevice / allocator / inert renderer / runtime callable

blackhole-py-rewrite/artifact.py
    TTBundleArtifact, TensorPort, ScalarPort, resource manifest, stable serializer/hash

blackhole-py-rewrite/tinygrad_lowering.py
    register lowerer
    recognize one TT pointwise root
    emit one complete PROGRAM/CALL/AFTER graph

blackhole-py-rewrite/templates/elementwise.py
    parameterize the existing add1 five-role sequence

tinygrad/test/unit/test_tt_graph_lowering.py
blackhole-py-rewrite/tests/test_artifact.py
blackhole-py-rewrite/tests/test_tinygrad_add1.py
```

The lowerer should decline every graph except the exact supported one-tile TT root. The artifact test should require stable bytes/hash, ordered ports, all five role images, exact CB/parameter ranges, and a semantic-source fingerprint. A mock runtime should verify ordered binding and reuse across changed addresses. Only after those tests pass should the device test launch add1.

UOp-level result:

```
# callified semantic root supplied to the lowerer
semantic = SINK(AFTER(out, STORE(out, x + 1)))

# lowerer result; artifact contains five role images and manifest
program = PROGRAM(semantic, empty LINEAR, manifest SOURCE, role BINARY,
                  ProgramInfo(globals=(0,1), outs=(0,), ins=(1,), ...))
call     = CALL(program, out, x)
result   = SINK(AFTER(out, call))
```

No generic scalar code renderer, TTIR, Tensor subclass, model-name match, or custom execution Op is required.

13.4 Patch acceptance commands and limits

The recorded host-only patch validation used the focused tests represented by:

```
cd tinygrad
PYTHONDONTWRITEBYTECODE=1 PYTHONHASHSEED=0 \
  python3 -m unittest -v test.unit.test_backend_graph_lowerer

# selected existing regressions were run explicitly; see the captured output
cd ../blackhole-py-rewrite
PYTHONDONTWRITEBYTECODE=1 PYTHONHASHSEED=0 \
  python3 -m unittest -v tests.test_program_bind
```

The evidence is the captured output, not the commands alone (research-output/architecture_hook_test.out:1-9; research-output/architecture_hook_selected_regression.out:1-14; research-output/first_patch_test.out:1-9). Both diffs are experimental patches against copies of the captured source; they have not been committed to either project.

13.5 Final recommendation

Land the generic post-callify graph hook first, land binding ABI hardening in parallel, and prove one complete add1 PROGRAM through a mock TT runtime before porting any large historical kernel. Then extend by the milestone order: masks/broadcasts, RMSNorm, matmul epilogues, RoPE/KV mutation, GQA/softmax/PV, SwiGLU/MLP, one unmodified block, and only then full model/runtime optimization.

The architectural invariant should remain simple: **Tinygrad owns semantics, callification, dependencies, buffer states, and execution orchestration; the TT lowerer owns tile/layout analysis, legal fusion, deterministic partitioning, resource verification, five-role artifact generation, and target-specific binding.** Every unsupported or cut region must remain an ordinary, inspectable Tinygrad graph decision with a source-grounded diagnostic.